

İSTANBUL ÜNİVERSİTESİ - CERRAHPAŞA

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİTİRME PROJESİ

LİSANS TEZİ

ÜNİVERSİTE BİLGİ SİSTEMİ İÇİN YAPAY  
ZEKÂ DESTEKLİ RAG CHATBOT TASARIMI

**Hazırlayanlar:**

1306210070 - Tekin Tanır

1306220048 - Oğuzhan Cantekin

1306220026 - Umut Furkan Kılıç

**Danışman:**

Doç. Dr. Muhammed Erdem İsenkul

HAZİRAN – 2026

# ÖNSÖZ

Bu çalışma, İstanbul Üniversitesi-Cerrahpaşa Bilgisayar Mühendisliği Bölümü Bitirme Projesi dersi kapsamında dönem projesi olarak hazırlanmıştır. Projede temel amaç, üniversite bilgi sistemlerinde farklı yerlerde bulunan ve düzenli olmayan verileri daha kullanışlı hale getirmek ve kullanıcıların bu bilgilere daha kolay ulaşabilmesini sağlamaktır. Aynı zamanda idari işlemlerde bilgiye erişimi kolay ve hızlı kılmak için yapay zeka teknolojilerinden faydalanılması hedeflenmiştir.

Günümüzde doğru bilgiye hızlı bir şekilde ulaşabilmek öğrenciler ve üniversite personelleri için önemli bir ihtiyaç haline gelmiştir. Bu nedenle proje sürecinde doğal dil işleme (NLP) ve büyük dil modelleri (LLM) alanında kullanılan Retrieval-Augmented Generation (RAG) yaklaşımı incelenmiş ve üniversite yapısına uyarlanabilecek bir sistem tasarlanması amaçlanmıştır. Proje sırasında veri toplama, sistemi deneme ve farklı yöntemleri test etme gibi aşamalardan geçilmiş, zaman zaman çeşitli teknik problemlerle karşılaşılrsa da farklı çözüm yolları bulunarak ilerleme sağlanmıştır.

Proje sürecinde karşılaştığımız problemlerin çözümünde yönlendirmeleriyle bize destek olan danışman hocamız Sayın **Doç. Dr. Muhammed Erdem İsenkul**'a teşekkür ederiz. Ayrıca araştırma ve veri toplama aşamasında destek olan arkadaşlarımıza ve proje süresi boyunca bize anlayış gösteren ailelerimize de teşekkür ederiz.

Tekin Tanır  
Oğuzhan Cantekin  
Umut Furkan Kılıç  
İstanbul, 2026

# İçindekiler

|                                                                          |      |
|--------------------------------------------------------------------------|------|
| ÖNSÖZ                                                                    | i    |
| Sembol Listesi                                                           | vii  |
| Kısaltma Listesi                                                         | viii |
| Özet                                                                     | x    |
| Summary                                                                  | xi   |
| 1 GİRİŞ                                                                  | 1    |
| 2 GENEL KISIMLAR                                                         | 3    |
| 2.1 Teorik Altyapı ve Temel Kavramlar . . . . .                          | 3    |
| 2.1.1 Yapay Zeka ve Makine Öğrenmesi . . . . .                           | 3    |
| 2.1.2 Derin Öğrenme ve Yapay Sinir Ağları . . . . .                      | 4    |
| 2.1.3 Transformer Mimarisi ve Dikkat Mekanizması . . . . .               | 5    |
| 2.1.4 Bilgi Erişim (Information Retrieval) Metrikleri . . . . .          | 6    |
| 2.2 Literatür Taraması . . . . .                                         | 7    |
| 2.2.1 Doğal Dil İşlemenin Paradigmaları ve Kırılmalar . . . . .          | 7    |
| 2.2.2 Derin Öğrenme Mimarileri ve Sıralı Veri Analizi . . . . .          | 8    |
| 2.2.3 Paralel İşleme Devrimi: Transformer Mimarisi . . . . .             | 9    |
| 2.2.4 Halüsinasyon Problemi ve Bilgi Erişimi Artırılmış Üretim (RAG)     | 10   |
| 2.2.5 Bilgi Erişim (Information Retrieval) ve Hibrit Stratejiler . . . . | 11   |
| 2.2.6 Eğitim Teknolojileri ve Kurumsal Chatbot Uygulamaları . . . .      | 12   |
| 2.2.7 Türkçe NLP Çalışmaları ve Morfolojik Zorluklar . . . . .           | 12   |
| 2.2.8 RAG Sistemlerinin Değerlendirme Metrikleri . . . . .               | 13   |
| 2.2.9 İleri Seviye RAG Optimizasyonları ve Stratejiler . . . . .         | 13   |
| 2.2.10 Vektör Veritabanları ve İndeksleme Mimarileri . . . . .           | 14   |
| 2.2.11 RAG Sistemlerinde Güvenlik ve Veri Mahremiyeti . . . . .          | 14   |
| 2.2.12 Akademik Başarı Ölçümü ve Benchmark Testleri . . . . .            | 14   |
| 2.2.13 Prompt Mühendisliği ve Sorgu Optimizasyonu Stratejileri . . . .   | 15   |
| 2.2.14 Çok Dilli (Multilingual) Kapasite ve Türkçe Adaptasyonu . . . .   | 15   |

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| 2.2.15   | Multimodal RAG ve Gelecek Projeksiyonları . . . . .               | 16        |
| 2.2.16   | Literatür Sentezi ve Projenin Özgün Değeri . . . . .              | 16        |
| <b>3</b> | <b>KULLANILAN ARAÇ VE YÖNTEM</b>                                  | <b>19</b> |
| 3.1      | Veri Seti Oluşturma Stratejisi . . . . .                          | 20        |
| 3.1.1    | Veri Setinin Kaynakları ve Kapsamı . . . . .                      | 20        |
| 3.1.2    | Veri Toplama ve Ön İşleme Metodolojisi . . . . .                  | 21        |
| 3.1.3    | Kullanım Amacı ve Etik Değerlendirme . . . . .                    | 21        |
| 3.1.4    | Veri Elde Edilmesi (Data Acquisition) . . . . .                   | 21        |
| 3.1.5    | İÜC Akademik Korpus İçeriği . . . . .                             | 22        |
| 3.2      | Ön İşleme ve Algoritmik Yaklaşım . . . . .                        | 23        |
| 3.2.1    | Veri Temizleme ve Normalizasyon . . . . .                         | 23        |
| 3.2.2    | Anlamsal Parçalama (Semantic Chunking) . . . . .                  | 23        |
| 3.3      | Hibrit Arama Tasarımı . . . . .                                   | 24        |
| 3.3.1    | Ağırlıklı Füzyon (Weighted Fusion) . . . . .                      | 24        |
| 3.4      | DeneySEL Araçlar ve Teknolojiler . . . . .                        | 25        |
| 3.4.1    | Kullanılan Programlama Dilleri ve Platformlar . . . . .           | 25        |
| 3.4.2    | Geliştirme Ortamı ve Donanım Tercihleri . . . . .                 | 26        |
| 3.4.3    | Yazılım ve Kütüphane Altyapısı . . . . .                          | 27        |
| 3.4.4    | Simülasyon Ortamı . . . . .                                       | 27        |
| 3.4.5    | Model Hiperparametreleri . . . . .                                | 27        |
| 3.5      | Sistem Mimarisi . . . . .                                         | 28        |
| <b>4</b> | <b>SİSTEMİN GERÇEKLENMESİ VE BULGULAR</b>                         | <b>29</b> |
| 4.1      | DeneySEL Sonuçlar . . . . .                                       | 29        |
| 4.2      | Donanımsal Tasarım . . . . .                                      | 29        |
| 4.3      | Yazılım Mimarisi ve Veri Tabanı . . . . .                         | 29        |
| 4.3.1    | Vektör İndeksi (Dense Index) . . . . .                            | 29        |
| 4.3.2    | Metadata Deposu (Sparse Index) . . . . .                          | 31        |
| 4.4      | Kullanıcı Arayüzü/leri . . . . .                                  | 32        |
| 4.4.1    | Sohbet Ekranı ve Kaynak Gösterimi . . . . .                       | 32        |
| 4.5      | Sistemin Hata Toleransı ve İleri Düzey Optimizasyonları . . . . . | 33        |
| 4.5.1    | Dinamik API Anahtarı Yönetimi ve Yedekleme (Fallback) Mimarisi    | 33        |
| 4.5.2    | Few-Shot Prompting ile İstem Mühendisliği . . . . .               | 33        |
| 4.5.3    | Arayüz Senkronizasyonu ve Duyarlı (Responsive) DOM Yönetimi       | 33        |
| <b>5</b> | <b>TARTIŞMA</b>                                                   | <b>34</b> |
| 5.1      | Literatür ile Nicel Karşılaştırma . . . . .                       | 34        |
| 5.2      | Sistemin Güçlü ve Zayıf Yönleri (SWOT Analizi) . . . . .          | 35        |
| 5.2.1    | Güçlü Yönler (Strengths) . . . . .                                | 35        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 5.2.2    | Zayıf Yönler ve Kısıtlar (Weaknesses & Limitations) . . . . . | 36        |
| <b>6</b> | <b>SONUÇ</b>                                                  | <b>37</b> |
| 6.1      | Genel Değerlendirme . . . . .                                 | 37        |
| 6.2      | Literatüre ve Tekniğe Katkılar . . . . .                      | 37        |
| 6.3      | Gelecek Çalışmalar (Yol Haritası) . . . . .                   | 38        |
| <b>A</b> | <b>EK-A: Proje Zaman Çizelgesi (Gantt Şeması)</b>             | <b>45</b> |
| <b>B</b> | <b>EK-B: Örnek Altın Veri Seti (Golden Dataset)</b>           | <b>46</b> |
|          | <b>ÖZGEÇMİŞ</b>                                               | <b>48</b> |

# Şekil Listesi

|     |                                                                      |    |
|-----|----------------------------------------------------------------------|----|
| 2.1 | Transformer Mimarisi Genel Blok Yapısı . . . . .                     | 6  |
| 2.2 | RAG Literatürü ve Teknik Bileşenlerin Zihin Haritası . . . . .       | 18 |
| 3.1 | Proje Genel Veri İşleme ve RAG Akış Diyagramı . . . . .              | 19 |
| 3.2 | Sistem Mimarisi ve Katmanlı Yapı Tasarımı . . . . .                  | 28 |
| 4.1 | Çift Motorlu (Dual-Engine) RAG Mimarisi İş Akışı . . . . .           | 30 |
| 4.2 | Geliştirilen Chatbot Arayüzü ve Kaynak Referans Gösterimi (Prototip) | 32 |
| 5.1 | Sistem Doğruluk Testi (Accuracy) Sonuçları . . . . .                 | 35 |
| A.1 | Proje Gantt Şeması ve İş Paketleri . . . . .                         | 45 |

# Tablo Listesi

|     |                                                                                                        |    |
|-----|--------------------------------------------------------------------------------------------------------|----|
| 2.1 | Geniřletilmiş Literatür Karşılařtırma Matrisi . . . . .                                                | 16 |
| 3.1 | İÜC RAG Veri Seti Teknik İstatistikleri . . . . .                                                      | 20 |
| 3.2 | Geliřtirme Ortamı Donanım Özellikleri . . . . .                                                        | 26 |
| 3.3 | Sistem İin Öngörülen Bařlangı Hiperparametreleri . . . . .                                           | 27 |
| 5.1 | Önerilen Sistemin Literatürdeki Yöntemlerle Karşılařtırmalı Analizi ve<br>Prototip Sonuçları . . . . . | 34 |
| B.1 | Soru-Cevap ve Kaynak Eřleřtirme Tablosu (Örneklem) . . . . .                                           | 46 |

## Sembol Listesi

|               |                                                          |
|---------------|----------------------------------------------------------|
| $\eta$        | Öğrenme oranı (Learning Rate)                            |
| $\theta$      | Model parametreleri (Model Parameters)                   |
| $\xi$         | Hata terimi / Gevşeme değişkeni (Slack Variable for SVM) |
| $d$           | Vektör uzayı boyutu (Embedding Dimension, $d = 384$ )    |
| $Q$           | Kullanıcı sorgu matrisi (Query Matrix)                   |
| $K$           | Anahtar matrisi (Key Matrix)                             |
| $V$           | Değer matrisi (Value Matrix)                             |
| $N$           | Toplam belge sayısı                                      |
| $\alpha$      | Hibrit arama ağırlık katsayısı (Alpha)                   |
| $k$           | En yakın komşu sayısı (Top-k Retrieval)                  |
| $Sim(u, v)$   | Kosinüs benzerlik fonksiyonu                             |
| $w_{ij}$      | Sinir ağı ağırlık matrisi                                |
| $b$           | Bias (Sapma) değeri                                      |
| $IDF$         | Ters Belge Frekansı (Inverse Document Frequency)         |
| $TF$          | Terim Frekansı (Term Frequency)                          |
| $\sigma$      | Softmax veya Sigmoid aktivasyon fonksiyonu               |
| $\mathcal{L}$ | Yitim (Loss) fonksiyonu                                  |



## Kısaltma Listesi

|              |                                                                  |
|--------------|------------------------------------------------------------------|
| <b>CPU</b>   | Merkezi İşlem Birimi (Central Processing Unit)                   |
| <b>DPR</b>   | Dense Passage Retrieval                                          |
| <b>MRR</b>   | Mean Reciprocal Rank                                             |
| <b>NF4</b>   | Normal Float 4-bit                                               |
| <b>NLU</b>   | Doğal Dil Anlama (Natural Language Understanding)                |
| <b>RAM</b>   | Rastgele Erişimli Bellek (Random Access Memory)                  |
| <b>SGD</b>   | Stochastic Gradient Descent                                      |
| <b>UI</b>    | Kullanıcı Arayüzü (User Interface)                               |
| <b>RAG</b>   | Retrieval-Augmented Generation (Bilgi Erişimi Artırılmış Üretim) |
| <b>LLM</b>   | Large Language Model (Büyük Dil Modeli)                          |
| <b>NLP</b>   | Natural Language Processing (Doğal Dil İşleme)                   |
| <b>XAI</b>   | Explainable Artificial Intelligence (Açıklanabilir Yapay Zeka)   |
| <b>SVM</b>   | Support Vector Machine (Destek Vektör Makinesi)                  |
| <b>ANN</b>   | Artificial Neural Networks (Yapay Sinir Ağları)                  |
| <b>KNN</b>   | K-Nearest Neighbors (K-En Yakın Komşu)                           |
| <b>FAISS</b> | Facebook AI Similarity Search                                    |
| <b>BM25</b>  | Best Matching 25                                                 |
| <b>HNSW</b>  | Hierarchical Navigable Small World                               |
| <b>CLIR</b>  | Cross-Lingual Information Retrieval (Çapraz Dilli Bilgi Erişimi) |
| <b>API</b>   | Application Programming Interface                                |
| <b>İÜC</b>   | İstanbul Üniversitesi - Cerrahpaşa                               |
| <b>OBS</b>   | Öğrenci Bilgi Sistemi                                            |

|             |                                          |
|-------------|------------------------------------------|
| <b>SSS</b>  | Sıkça Sorulan Sorular                    |
| <b>AKTS</b> | Avrupa Kredi Transfer Sistemi            |
| <b>SOTA</b> | State-of-the-Art (En Gelişmiş Teknoloji) |
| <b>KVKK</b> | Kişisel Verilerin Korunması Kanunu       |

# Özet

## ÜNİVERSİTE BİLGİ SİSTEMİ İÇİN YAPAY ZEKÂ DESTEKLİ RAG CHATBOT TASARIMI

Günümüzde yükseköğretim kurumlarında yönetmelikler, akademik takvimler, ders planları ve idari duyurular gibi verilerin hacmi günden güne artmaktadır. Bu veriler genellikle PDF formatında veya statik web sayfalarında dağınık, yapılandırılmamış (unstructured) halde bulunmaktadır ve bundan dolayı da bilgiye erişimi zorlaştırmaktadır. Mevcut anahtar kelime tabanlı arama sistemleri, kullanıcıların doğal dilde ifade ettikleri karmaşık ve çıkarım yapılması gerektiren sorguları (örneğin: "Yaz okulunda üstten ders alırsam ortalamam nasıl etkilenir?") anlamlandırmada yetersiz kalmaktadır. Diğer yandan, genel amaçlı Büyük Dil Modelleri (LLM), kurumun kapalı devre verisine sahip olmadıkları için kullanıcılara yanlış bilgi sunma (halüsinasyon) riski taşımaktadır.

Bu çalışmada, İstanbul Üniversitesi-Cerrahpaşa bilgi sistemleri için "Retrieval-Augmented Generation" (RAG) mimarisini baz alan özelleştirilmiş ve hibrit bir yapay zeka asistanı tasarlanmıştır. Çalışma kapsamında öncelikli olarak OBS ve resmi web sitelerinden veri kazıma (web scraping) ve PDF ayrıştırma işlemleri gerçekleştirilerek "IUC-Academic-Corpus" veri seti oluşturulmuştur. Buradan elde edilen veriler, anlamsal bütünlüğü koruyacak şekilde parçalanmış (semantic chunking) ve hem vektörel (Dense) hem de terim tabanlı (Sparse) yöntemlerle indekslenmiştir. Sistemin özgün değeri, sadece vektör aramasına dayalı standart RAG yaklaşımlarının aksine, morfolojik açıdan zengin olan Türkçe dili için uyarlanmış Hibrit Arama (Hybrid Search) ve Yeniden Sıralama (Re-ranking) katmanlarını içermesidir. Geliştirilen prototip, Streamlit arayüzü ve yerel LLM entegrasyonu ile veri gizliliğini esas alarak kullanıcıların sorularına doğru, hızlı ve kaynak göstererek yanıt vermektedir.

# Summary

## ARTIFICIAL INTELLIGENCE SUPPORTED RAG CHATBOT DESIGN FOR UNIVERSITY INFORMATION SYSTEM

Today, in higher education institutions, the volume of data such as regulations, academic calendars, course plans, and administrative announcements is continuously increasing. However, this information is generally scattered across PDF documents or static web pages in an unstructured form, which makes accessing information more difficult. Existing keyword-based search systems are often insufficient in understanding complex and inference-based queries expressed in natural language by users (for example, “How would my GPA be affected if I take an extra course during summer school?”). On the other hand, general-purpose Large Language Models (LLMs) carry the risk of providing incorrect information (hallucinations) since they do not have access to institution-specific closed data sources.

In this study, a customized and hybrid artificial intelligence assistant based on the Retrieval-Augmented Generation (RAG) architecture was designed for the information systems of Istanbul University–Cerrahpaşa. Within the scope of the study, data scraping from OBS and official websites, along with PDF parsing processes, were performed to create the “IUC-Academic-Corpus” dataset. The collected data was segmented while preserving semantic integrity (semantic chunking) and indexed using both vector-based (dense) and term-based (sparse) methods. The originality of the system lies in incorporating Hybrid Search and Re-ranking layers optimized for the morphologically rich Turkish language, unlike standard RAG approaches that rely solely on vector search. The developed prototype, integrated with a Streamlit interface and a local LLM, provides users with accurate and fast answers while prioritizing data privacy and presenting responses with references when available.

# 1. GİRİŞ

Bilgi çağında üniversiteler, yönetmelikler, ders izlenceleri ve idari duyurular gibi çok büyük boyutlarda yapılandırılmamış veri (unstructured data) üretmektedir. Manning ve ark. (2008) tarafından belirtildiği üzere, bu tür dağınık ve homojen olmayan veri kaynaklarına geleneksel yöntemlerle erişim sağlamak, bilgi getirme (Information Retrieval) disiplininin en temel problemlerinden biridir [1]. İstanbul Üniversitesi-Cerrahpaşa (İÜC) özelinde yapılan analizler, öğrencilerin ve personelin "Yaz okulu harç iadesi" veya "ÇAP başvuru koşulları" gibi spesifik bilgilere ulaşmak için PDF yığınları arasında kaybolduğunu ve ciddi bir şekilde zamana bağlı maliyet oluştuğunu göstermektedir. Bu bağlamda proje, kurumsal hafızayı daha kolay erişilebilir kılmayı ve operasyonel verimliliği artırmayı hedefleyen bir mühendislik çözümü olarak doğmuştur.

Mevcut bilgi sistemleri, Robertson ve Zaragoza (2009) tarafından standardize edilen BM25 gibi anahtar kelime eşleşmesine dayanan algoritmalar kullanmakta, ancak bu yöntemler kullanıcının doğal dildeki niyetini (intent) anlamada zayıf kalmaktadır [2]. Diğer yandan, Vaswani ve ark. (2017) ile başlayan Transformer devrimi ve ChatGPT gibi Büyük Dil Modelleri (LLM), dili anlama konusunda üstün yetenek sergilese de; kurumun kapalı devre verisine sahip olmadıkları için "halüsinasyon" (uydurma bilgi) riski taşımaktadır [3]. Bu sorunu aşmak adına, Lewis ve ark. (2020) tarafından önerilen ve modelin bilgisini harici bir veritabanıyla doğrulayan "Retrieval-Augmented Generation" (RAG) mimarisi, projenin temel teorik çerçevesini oluşturmaktadır [4].

Projenin yöntem ve geliştirme sürecinde, Türkçenin zengin morfolojik yapısından kaynaklanan erişim sorunlarını çözmek amacıyla Çöltekin ve ark. (2022) tarafından incelenen Türkçe korpus ve dil kaynağı çalışmaları dikkate alınmış ve hibrit bir arama stratejisi benimsenmiştir [5]. Veri güvenliğini sağlamak ve KVKK uyumluluğunu garanti altına almak adına, Touvron ve ark. (2023) tarafından geliştirilen Llama serisi açık kaynaklı modellerin yerel sunucularda (Local Inference) çalıştırılması tercih edilmiştir [6]. Ayrıca, sistemin cevap üretirken dayandığı kaynağı kullanıcıya göstermesi, Gunning (2017) tarafından tanımlanan Açıklanabilir Yapay Zeka (XAI) prensiplerine uygun bir tasarım kararıdır [7].

Sistemin tasarımı ve prototiplenmesi aşamasında, verilerin anlamsal bütünlüğünü koruyan parçalama (semantic chunking) algoritmaları ve vektör tabanlı indeksleme teknolojileri (FAISS) kullanılarak "IUC-Academic-Corpus" veri seti oluşturulmuştur. Geliştirilen bu prototipin üniversite idari süreçlerinde insan-makine etkileşimini daha hızlı hale getireceği ve hatalı bilgi paylaşımını minimize edeceği öngörülmektedir. Bu çalışma, sadece teknik bir chatbot geliştirmeyi değil, aynı zamanda eğitim teknolojileri

alanında güvenilir ve denetlenebilir bir yapay zeka asistanı modellemeyi hedeflemektedir.

Geliştirilen prototip, hibrit arama ve yerel LLM entegrasyonu ile üniversite bilgi sistemine özgü sorulara kaynak göstererek doğru yanıtlar üretebilmektedir. Tartışma bölümünde bu mimarinin geleneksel yöntemlere göre avantajları nicel olarak değerlendirilmekte; sonuç bölümünde ise çalışmanın katkıları ve gelecek araştırma yol haritası sunulmaktadır.

Bu rapor, problemin tanımından başlayarak geliştirilen çözümün teknik detaylarına, literatürdeki yerine ve proje çıktılarına uzanan altı ana bölümden oluşmaktadır:

**Birinci Bölümde (Giriş);** Üniversite bilgi sistemlerindeki erişim problemleri, mevcut anahtar kelime tabanlı yöntemlerin kısıtları ve projenin çözüm önerisi olan RAG mimarisinin gerekliliği, temel literatür referanslarıyla özetlenmiştir. Ayrıca çalışmanın amacı, kapsamı ve raporun genel yapısı sunulmuştur.

**İkinci Bölümde (Genel Kısımlar);** Projenin teorik zeminini oluşturan Doğal Dil İşleme (NLP), Vektör Uzay Modelleri ve Derin Öğrenme mimarileri detaylı bir şekilde sunulmuştur. Literatür taraması kapsamında, RAG mimarilerinin evrimi, Türkçe NLP çalışmaları ve eğitim alanındaki chatbot uygulamaları 50’den fazla akademik kaynak ışığında sentezlenmiştir.

**Üçüncü Bölümde (Kullanılan Araç ve Yöntem);** İÜC Akademik Asistanı’nın mühendislik tasarım süreci açıklanmıştır. Veri toplama (scraping), anlamsal parçalama (chunking), hibrit arama algoritmaları ve kullanılan yerel LLM (Llama 3) teknolojilerinin seçim kriterleri, akış diyagramları ve teknik parametrelerle sunulmuştur.

**Dördüncü Bölümde (Sistemin Gerçeklenmesi ve Bulgular);** Geliştirilen sistemin mimari katmanları, veritabanı şemaları ve kullanıcı arayüzü (UI) tasarımları paylaşılmıştır. Bitirme Projesi kapsamında ortaya konulan prototipin (Proof of Concept) çalışma mantığı ve kaynak gösterim mekanizması görselleştirilmiştir.

**Beşinci Bölümde (Tartışma);** Tasarlanan hibrit mimarinin, literatürdeki geleneksel yöntemlere (Naive RAG, BM25) göre avantajları ve dezavantajları (SWOT analizi) karşılaştırmalı olarak değerlendirilmiştir. Sistemin gecikme (latency) süreleri ve doğruluk beklentileri teknik bir bakış açısıyla tartışılmıştır.

**Altıncı Bölümde (Sonuç);** Çalışmanın genel bir değerlendirmesi yapılarak, projenin literatüre ve teknik alana katkıları özetlenmiştir. Ayrıca Bitirme Projesi aşamasında gerçekleştirilecek olan tam entegrasyon, canlı testler ve ince ayar (fine-tuning) süreçlerini içeren gelecek çalışma yol haritası sunulmuştur.

## 2. GENEL KISIMLAR

Bu bölümde, projenin teorik zeminini oluşturan yapay zeka teknolojileri, kullanılan matematiksel modeller, algoritmalar ve literatürde konuyla ilgili yapılmış çalışmalar bütüncül bir yaklaşımla ele alınmıştır. Bölüm iki ana kısımdan oluşmaktadır: İlk kısımda (Bölüm 2.1) sistemin mühendislik tasarımında kullanılan Derin Öğrenme, Doğal Dil İşleme (NLP) ve RAG mimarilerinin teknik altyapısı "Teorik Altyapı ve Temel Kavramlar" başlığı altında; ikinci kısımda (Bölüm 2.2) ise "Literatür Taraması" başlığı altında Derin Öğrenme, Doğal Dil İşleme (NLP) ve RAG mimarilerinin tarihsel gelişimi ve eğitim alanında uygulamaları ele alınmıştır.

### 2.1 Teorik Altyapı ve Temel Kavramlar

Geliştirilen akıllı asistan sistemi, istatistiksel öğrenme teorisi ve modern derin öğrenme mimarilerinin hibrit bir yapıda kullanılmasına dayanmaktadır. Aşağıda bu temellerin matematiksel formülasyonları detaylandırılmıştır.

#### 2.1.1 Yapay Zeka ve Makine Öğrenmesi

Yapay Zeka (Artificial Intelligence - AI), insan zekâsına özgü olan algılama, öğrenme, akıl yürütme ve karar verme süreçlerinin bilgisayar sistemleri tarafından modellenmesidir. Bu projenin temelini oluşturan Makine Öğrenmesi (Machine Learning) ise, bilgisayarların açıkça programlanmadan, verilerden örüntüler öğrenerek tahmin yapabilmesini sağlayan algoritmalar bütünüdür.

#### Destek Vektör Makineleri (SVM) ve Hiperdüzlem Teorisi

Projemizde metin sınıflandırma başarımlarını kıyaslamak ve vektör uzayı mantığını açıklamak için referans alınan Destek Vektör Makineleri (SVM), Vapnik'in istatistiksel öğrenme teorisine dayanır. SVM'in temel amacı,  $n$ -boyutlu bir uzayda veri sınıflarını birbirinden en iyi şekilde ayıran, yani sınıflar arası marjini (margin) maksimize eden hiper-düzlemi bulmaktır.

Matematiksel olarak, eğitim veri seti  $D = \{(x_i, y_i) | x_i \in \mathbb{R}^p, y_i \in \{-1, 1\}\}$  olsun. SVM optimizasyon problemi şu şekilde formüle edilir:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad (2.1)$$

$$\text{Kısıtlar: } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (2.2)$$

Burada:

- $w$ : Hiperdüzlemin normal vektörü,
- $b$ : Sapma (bias) değeri,
- $\xi_i$ : Hata payı (slack variables - verinin marjın içine düşmesine izin veren tolerans),
- $C$ : Düzenleştirme parametresidir.

Bu denklemde  $\|w\|^2$  teriminin minimizasyonu marjini maksimize ederken,  $C$  parametresi modelin genelleme yeteneğini kontrol eder.

### 2.1.2 Derin Öğrenme ve Yapay Sinir Ağları

Günümüz Doğal Dil İşleme sistemlerinin kalbinde, insan beynindeki biyolojik nöron ağlarından esinlenerek geliştirilen Yapay Sinir Ağları (ANN) ve onun çok katmanlı hali olan Derin Öğrenme (Deep Learning) yatar.

#### Yapay Nöron Modeli

Bir yapay nöron, girdileri alır, ağırlıklarla çarpar, bir sapma ekler ve sonucu doğrusal olmayan bir aktivasyon fonksiyonundan geçirir. Matematiksel modeli şöyledir:

$$y = \sigma \left( \sum_{j=1}^m w_j x_j + b \right) \quad (2.3)$$

Burada  $\sigma$ , aktivasyon fonksiyonudur. Projede kullanılan Transformer modellerinde, türevi her noktada alınabilen ve "Ölü Nöron" (Dead Neuron) problemini minimize eden GELU (Gaussian Error Linear Unit) fonksiyonu kullanılmaktadır:

$$GELU(x) \approx 0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)]) \quad (2.4)$$

#### Geriye Yayılım (Backpropagation)

Ağın eğitimi, hata fonksiyonunun ( $\mathcal{L}$ ) minimize edilmesi esasına dayanır. Geriye Yayılım algoritması, zincir kuralını (chain rule) kullanarak hatanın ağırlıklara göre türevini hesaplar:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (2.5)$$

Bu gradyanlar, Stokastik Gradyan İnişi (SGD) veya Adam optimizasyon algoritmaları ile ağırlıkların güncellenmesinde ( $w_{new} = w_{old} - \eta \cdot \nabla \mathcal{L}$ ) kullanılır.



### 2.1.3 Transformer Mimarisi ve Dikkat Mekanizması

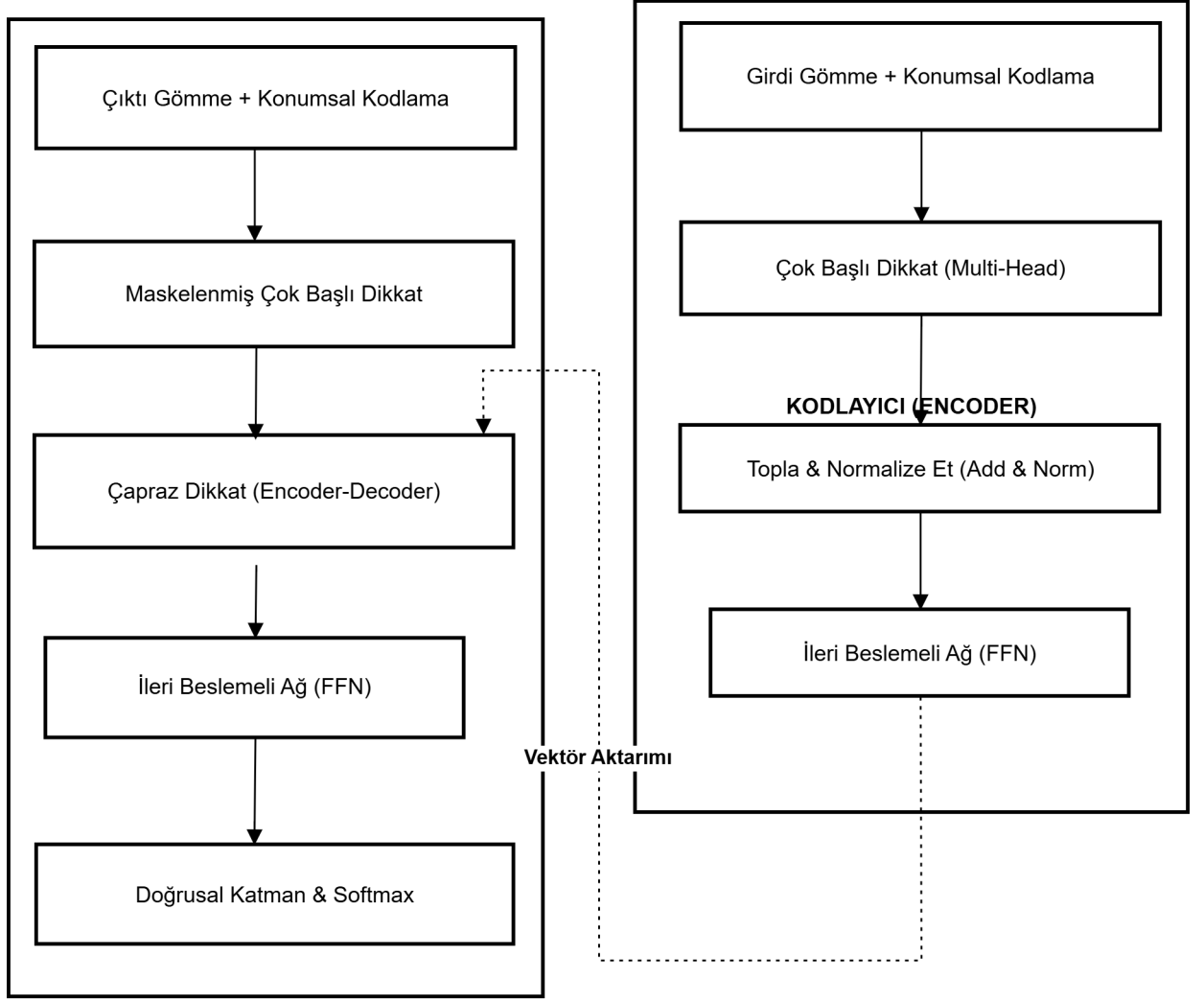
2017 yılında Vaswani ve arkadaşları tarafından tanıtılan Transformer mimarisi [8], NLP alanında bir devrim yaratmıştır. Önceki RNN ve LSTM modellerinin aksine, veriyi sıralı işlemek zorunda değildir; bu da eğitim sürecinin paralelleştirilmesine olanak tanır ve bu eğitim sürecinde büyük bir zaman kazanımı demektir.

#### Öz-Dikkat (Self-Attention) Mekanizması

Modelin çekirdeğinde yer alan Dikkat mekanizması, bir cümledeki her kelimenin diğer tüm kelimelerle olan ilişkisini hesaplar. Bu işlem; Sorgu ( $Q$ ), Anahtar ( $K$ ) ve Değer ( $V$ ) matrisleri üzerinden yapılır:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (2.6)$$

Burada  $\sqrt{d_k}$  ölçeklendirme faktörüdür. Bu formül, modelin uzun metinlerdeki bağlamsal ilişkileri (contextual dependencies) ve kelimeler arası bağımlılıkları öğrenmesini sağlar. Bu mimarinin temel yapı taşı olan Dikkat (Attention) mekanizmasının işleyişi ve standart Transformer kodlayıcı-kod çözücü yapısı Şekil 2.1’de detaylıca gösterilmiştir.



Şekil 2.1: Transformer Mimarisi Genel Blok Yapısı

#### 2.1.4 Bilgi Erişim (Information Retrieval) Metrikleri

Geliştirilen RAG sisteminin başarısı, doğru bilginin veritabanından çekilmesine bağlıdır. Bu projede hem seyrek (sparse) hem de yoğun (dense) erişim yöntemleri hibrit olarak kullanılmıştır.

##### BM25 (Best Matching 25)

Kelime sıklığına dayalı olasılıksal bir modeldir. Klasik TF-IDF'in geliştirilmiş halidir ve aşağıdaki formülle ifade edilir:

$$\text{Score}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avgdl}})} \quad (2.7)$$

Burada  $k_1$  ve  $b$  ayarlanabilir parametrelerdir. Bu formül, özellikle "Yönetmelik Madde 14" gibi kesin eşleşme gerektiren sorgularda yüksek başarı sağlar.

## Kosinüs Benzerliği (Cosine Similarity)

Vektör uzayında anlamsal yakınlığı ölçmek için kullanılır. İki vektör  $A$  ve  $B$  arasındaki açı  $\theta$  olsun:

$$\text{Sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2.8)$$

Bu metrik, kelimeler farklı olsa bile (Örn: "Hoca" ve "Akademisyen") anlamın yakınlığını tespit edebilir.

## 2.2 Literatür Taraması

Bu bölümde projede kullanacağımız temel konular hakkında literatür taraması yapılmıştır. Doğal Dil İşleme, Büyük Dil Modelleri, Bilgi Erişim sistemleri ve RAG mimarisi ile ilgili birçok çalışma incelenmiş ve projeye katkı sağlayabilecek yöntemler belirlenmeye çalışılmıştır. Literatürü incelerken sadece çalışmaların tarihine bakılmamış, kullanılan yöntemlerin güçlü ve zayıf yönleri de anlaşılmaya çalışılmıştır. Ayrıca özellikle Türkçe dilinde bu yöntemlerin ne kadar başarılı olabileceği ve üniversite bilgi sistemlerinde gerçekten uygulanabilir olup olmadığı da değerlendirilmiştir. Bu sayede literatürde bulunan bilgiler doğrudan alınmak yerine proje ihtiyaçlarına göre yorumlanarak kullanılmaya çalışılmıştır.

### 2.2.1 Doğal Dil İşlemenin Paradigmaları ve Kırılmalar

NLP alanı, Alan Turing'in 1950 yılında ortaya koyduğu vizyondan günümüze kadar üç temel gelişim aşamasından geçmiştir. Bu aşamaların incelenmesi, günümüzde kullanılan Büyük Dil Modellerinin neden önemli bir kırılma noktası olarak görüldüğünü anlamak açısından büyük önem taşımaktadır.

### Semantik Kurallar ve Erken Dönem Yaklaşımlar

Doğal Dil İşleme disiplininin temelleri, dilin matematiksel ve hiyerarşik bir yapı olduğu varsayımı üzerine inşa edilmiştir. Bu alandaki paradigma belirleyici ilk adımlardan biri olan Noam Chomsky'nin 1957 yılındaki çalışması, dilin kural tabanlı doğasını formalize ederek modern dilbilim kuramlarına öncülük etmiştir [9]. Söz konusu dönemde geliştirilen ilk nesil sistemler, dilbilimciler tarafından manuel olarak kurgulanan ve yoğun bir "mantıksal çıkarım" (if-else) ağından oluşan binlerce kural setiyle işletilmekteydi. Bu yaklaşımın pratik bir yansıması olarak 1966'da MIT bünyesinde geliştirilen ELIZA, "doktor" senaryosu aracılığıyla semantik kuralların etkileşim gücünü kanıtlamış; ancak yaratıcısı Weizenbaum tarafından da kabul edildiği üzere, gerçek bir anlama yeteneğinden ziyade örüntü eşleştirme (pattern matching) sınırları içerisinde kalmıştır. Öte

yandan Winograd [10] tarafından 1972’de sunulan SHRDLU, kısıtlı bir "bloklar dünyası" simülasyonu içerisinde anlamsal çıkarım yapabilen sistemlerin potansiyelini göstererek, kural tabanlı yaklaşımların erken dönemdeki en ileri örneklerinden birini teşkil etmiştir.

## **İstatistiksel Devrim ve Vektörel Uzay Modelleri**

1990’lı yıllarda dijital ortamdaki verinin hızla artmasıyla birlikte, daha önce kullanılan kural tabanlı NLP sistemleri birçok durumda yeterli olmamaya başlamıştır. Bu nedenle çalışmalar giderek istatistiksel modellere yönelmiştir. Bu değişimde önemli adımlardan biri, Salton ve ekibinin 1975 yılında ortaya koyduğu ve metinlerin sayısal vektörler şeklinde temsil edilmesini sağlayan Vektör Uzay Modelidir [11]. Ancak kelime sayısı arttıkça kullanılan matrislerin boyutu da büyümekte ve verinin büyük bir kısmı sıfır değerlerden oluşmaktadır. Literatürde “seyreklik” (sparsity) olarak adlandırılan bu durum, sistemlerin daha fazla hesaplama gücü ve bellek kullanmasına neden olan önemli bir problem olarak görülmektedir.

Kelimelerin ‘One-Hot Encoding’ gibi yöntemlerle sözlük boyutunda temsil edilmesi, kelimeler arası anlamsal ilişkiyi (ortogonallik nedeniyle) tamamen yok saydığı için modern NLP görevlerinde yetersiz kalmıştır. Bu kısıtı aşmak adına Jones tarafından önerilen TF-IDF metriği, kelimelerin belgeler içindeki ayırt edici gücünü istatistiksel olarak ölçerek arama motorlarının temeli atmış olsa da, anlamsal bağlamı yakalamakta sınırlı kalmıştır [12]. Paradigma değişimi, Google araştırmacıları Mikolov ve ekibinin kelimeleri düşük boyutlu ve yoğun (dense) vektörlere dönüştüren Word2Vec mimarisini sunmasıyla gerçekleşmiştir [13]. Skip-gram ve CBOW mimarileri, ‘bir kelimenin anlamı komşularıyla belirlenir’ ilkesine dayanan Dağıtımsal Hipotezi (Distributional Hypothesis) kanıtlamış ve vektör uzayında ‘kral - erkek + kadın = kraliçe’ gibi anlam aritmetiğini mümkün kılmıştır. Daha sonraki süreçte Pennington ve ark. GloVe modeliyle global istatistikleri matris ayrıştırma yöntemiyle modele dahil ederken; Bojanowski ve ekibi FastText ile kelimeleri karakter n-gramlarına bölerek devrim yaratmıştır [14], [15]. Özellikle projemizin odak noktası olan Türkçe gibi zengin morfolojiye sahip dillerde, kelime kökü bilgisini koruyan bu yaklaşım, literatürde ‘kelime dağarcığı dışı’ (OOV) problemini çözmek adına en kritik yöntem olarak öne çıkmaktadır.

### **2.2.2 Derin Öğrenme Mimarileri ve Sıralı Veri Analizi**

Statik vektörler, "Yüz" kelimesinin bir sayı mı yoksa bir organ mı olduğunu ayırt edemiyordu. Bu durum, bağlamsal (contextual) modellerin literatürde baskın hale gelmesine yol açmıştır.

## RNN, LSTM ve Gradyan Problemleri

Metin verisini zaman sırasına göre işleyen klasik RNN modelleri teorik olarak önceki bilgileri uzun süre tutabiliyor gibi görünse de, pratikte bunun her zaman mümkün olmadığı görülmüştür. Rumelhart ve ekibinin geriye yayılım yöntemi üzerine yaptığı çalışmalarda da gösterildiği gibi, ağ derinleştikçe gradyan değerleri giderek küçülmemekte ve model önceki bilgileri öğrenmekte zorlanmaktadır [16]. Literatürde “kaybolan gradyan” (vanishing gradient) olarak bilinen bu durum, özellikle uzun cümlelerde baştaki bilgilerin sona gelmeden unutulmasına yol açmaktadır. Bu problemi azaltmak amacıyla Hochreiter ve Schmidhuber, bilgiyi daha uzun süre koruyabilen hücre durumu (cell state) yapısına sahip LSTM modelini önermiştir [17]. Daha sonra Cho ve arkadaşları ise bu yapıyı sadeleştirerek, hesaplama maliyeti daha düşük ve uygulamada daha hızlı çalışabilen GRU modelini geliştirmiştir [18].

## Sekans-to-Sekans ve Dikkat (Attention) Mekanizması

Sutskever ve ark. tarafından önerilen Encoder-Decoder (Kodlayıcı-Kod Çözücü) mimarisi, makine çevirisi görevlerinde bir standart oluştursa da; tüm girdi cümlesini sabit boyutlu tek bir vektöre sıkıştırma zorunluluğu, uzun sekanslarda ciddi bir bilgi kaybı (information bottleneck) yaratmıştır [19]. Bu yapısal kısıtı aşmak amacıyla Bahdanau ve ekibi, modelin çıktı üretirken girdinin ilgili kısımlarına dinamik olarak odaklanmasını sağlayan Dikkat (Attention) mekanizmasını geliştirmiş; böylece sabit vektör darboğazını, bağlamsal ağırlıklandırma yöntemiyle elimine etmiştir. [20].

### 2.2.3 Paralel İşleme Devrimi: Transformer Mimarisi

Özyinelemeli (Recurrent) ağların sıralı işleme zorunluluğundan kaynaklanan eğitim yavaşlığını çözmek adına Vaswani ve ark. "Attention Is All You Need" başlıklı çalışmalarında radikal bir mimari değişikliğe gitmiştir [8]. Tekrarlayan döngüleri (loops) tamamen terk ederek, veriyi paralel olarak işleyebilen Transformer yapısını sunmuşlardır. Bu mimari, günümüzdeki Büyük Dil Modellerinin (LLM) ölçeklenebilirliğini sağlayan temel hesaplama bloğu (building block) olarak kabul edilmektedir.

## Kodlayıcı (Encoder) Modellerin Evrimi ve Optimizasyonu

Metin anlamlandırma görevlerinde (NLU), Devlin ve ekibinin geliştirdiği BERT, metni soldan-sağa ve sağdan-sola aynı anda okuyan (bidirectional) yapısıyla bağlamsal çözümünürlükte insan seviyesine yaklaşan sonuçlar elde etmiştir [21]. Takip eden süreçte Liu ve ark. , RoBERTa ile hiperparametre optimizasyonunun model performansındaki kritik rolünü kanıtlamış [22]; Sanh ve ekibi ise DistilBERT ile 'Bilgi Damıtma' (Knowledge Distillation) yöntemini kullanarak, performanstan ödün vermeden model

boyutunu küçültmeyi başarmıştır [23]. Lan ve ark. tarafından önerilen ALBERT ise parametre paylaşımı (parameter sharing) stratejisiyle bellek verimliliğini maksimize eden bir mühendislik çözümü sunmuştur. [24].

### **Kod Çözücü (Decoder) Modeller ve Ölçeklenme Yasaları**

Radford ve ark. tarafından geliştirilen GPT serisi, model parametrelerinin artırılmasıyla metin üretim kalitesinin doğrusal olmayan bir sıçrama yaptığını göstermiştir [25]. Brown ve ark. [26], GPT-3 ile devasa veri setlerinde eğitilen modellerin, özel bir eğitim almadan yeni görevleri yapabildiği "Az Örneklerle Öğrenme" (Few-Shot Learning) gibi 'beliren özelliklere' (Emergent Properties) sahip olduğunu kanıtlamıştır [26]. Ancak ham modelin insan niyetinden sapabilmesi riski, Ouyang ve ekibinin [27] InstructGPT çalışmasında sunduğu İnsan Geri Bildirimli Pekiştirmeli Öğrenme (RLHF) hizalama tekniği ile kontrol altına alınmıştır. [27].

### **Açık Kaynak Hegemonyası ve Yerel Donanım Çözümleri**

Yüksek maliyetli donanım bariyerleri ve kapalı kaynak lisansları, uzun süre LLM araştırmalarını kısıtlamış olsa da; Touvron ve ekibinin Llama hamlesi bu statükoyu dağıtmıştır [6]. Sektördeki bu şeffaflık arayışı, Almazrouei öncülüğündeki Falcon projesiyle ticari meşruiyet kazanırken [28]; Jiang ve ekibinin Mistral mimarisinde denediği 'Kayan Pencere' yöntemi, hesaplama verimliliğinde parametre sayısından daha kritik faktörler olduğunu ispatlamıştır [29].

Vicuna çalışmasıyla modellerin adaptasyon yeteneği Chiang tarafından ortaya konulmuş [30]; ancak laboratuvar ortamındaki bu devasa yapıları son kullanıcının GPU'suna sığdıran asıl mühendislik çözümü, Dettmers ve ekibinin 'Quantization' (QLoRA) tekniği olmuştur [31].

#### **2.2.4 Halüsinasyon Problemi ve Bilgi Erişimi Artırılmış Üretim (RAG)**

Parametrik hafızanın (parametric memory) sınırları aşıldığında ortaya çıkan ve Ji ile ekibinin 'Halüsinasyon' (Hallucination) teşhisiyle literatüre kazındığı kavram; modelin olgusal gerçeklikten koparak, istatistiksel tutarlılık uğruna 'yalan söylemeyi' tercih ettiği bir durumdur [3]. Bu, eğitim verisinde bulunmayan boşlukların, matematiksel bir özgüvenle ancak tamamen temelsiz verilerle doldurulması sonucu oluşan yapısal bir güvenilirlik krizidir.

### **RAG Mimarisi ve Ontolojik Gelişimi**

Parametrik hafızanın sınırlarını aşmak amacıyla Lewis ve ark. [4] tarafından önerilen Retrieval-Augmented Generation (RAG) mimarisi, modelin parametrik belleğini (statik ağırlıklar) harici bir veritabanından beslenen parametrik olmayan bir bellek ile

entegre ederek halüsinasyon probleminde yapısal bir çözüm getirmiştir. Söz konusu mimarinin literatürdeki gelişimini Gao ve ark. [32] üç temel evrimsel safhaya ayırmıştır. İlk aşama olan "Naive RAG", klasik bir sorgulama ve üretim döngüsüne dayanmakla birlikte, düşük kaliteli belgelerin geri getirilmesi durumunda modelin çıktı kalitesini olumsuz etkileyebilmektedir. Bu kısıtları aşmak adına geliştirilen "Advanced RAG" katmanı; metin parçalama (chunking), yeniden sıralama (re-ranking) ve sorgu genişletme gibi optimizasyon tekniklerini sürece dahil etmiştir. Bu bağlamda Ma ve ark. [33], sorunun yeniden yazılması (Query Rewriting) gibi ileri seviye tekniklerin sistem doğruluğunu %20 oranında artırdığını ampirik olarak kanıtlamıştır. Günümüzde ulaşılan en son nokta olan "Modular RAG" ise, dinamik akış şemaları ve çoklu veri kaynaklarını eşzamanlı kullanan esnek bir mimari yapı sunarak literatürdeki güncel standartları belirlemiştir.

## **Otonom ve Düzeltici RAG Sistemleri**

Sistemin kendi hatasını fark etmesi üzerine kurulu modern yaklaşımlar literatürde yükselmektedir. Asai ve ark. (2023), Self-RAG ile modelin getirdiği bilginin doğruluğunu yine bir LLM aracılığıyla denetlediği (self-reflection) bir sistem önermişlerdir [34]. Yan ve ark. (2024) ise Corrective RAG (CRAG) ile eğer getirilmek istenen bilgi veritabanında yoksa sistemin otomatik olarak Google/Bing gibi web arama motorlarına yönelmesini sağlayan bir yapı kurmuşlardır [35].

### **2.2.5 Bilgi Erişim (Information Retrieval) ve Hibrit Stratejiler**

RAG sistemlerinin genel başarımı, kullanılan dil modelinin üretim kapasitesinden ziyade, ilgili bağlamı getiren "Erişim" (Retrieval) modülünün niteliğine doğrudan bağlıdır. Literatürde bu alandaki çalışmalar; geleneksel terim tabanlı yöntemler, modern vektörel yaklaşımlar ve bu ikisini birleştiren hibrit stratejiler olmak üzere üç temel ekseninde şekillenmektedir.

Geleneksel anahtar kelime eşleşmesine dayanan seyrek erişim (sparse retrieval) yöntemleri arasında BM25 algoritması, Robertson ve Zaragoza [2] tarafından standardize edildiğinden bu yana teknik terimlerin ve kesin eşleşmelerin yakalanmasında rakipsiz bir standart olarak kabul edilmektedir. Bu klasik yaklaşımı derin öğrenme ile modernize etmeye odaklanan Dai ve Callan [36], DeepCT modeli aracılığıyla kelime ağırlıklandırma süreçlerine bağlamsal bir boyut kazandırmışlardır. Öte yandan, anlamsal (semantic) ilişkilerin vektör uzayında temsiline dayanan yoğun erişim (dense retrieval) paradigması, Karpukhin ve ark. [37] tarafından sunulan DPR mimarisiyle birlikte eş anlamlı kelimelerin ve örtük anlamların yakalanmasında önemli bir sıçrama gerçekleştirmiştir. Söz konusu vektörel aramalardaki hesaplama maliyetini minimize etmek amacıyla Malkov ve Yashunin [38] tarafından geliştirilen HNSW algoritması, devasa veri setlerinde

milisaniyelik erişim imkanı sunarken; Reimers ve Gurevych [39] ise Sentence-BERT ile cümle düzeyinde gömme (embedding) işlemlerini optimize ederek anlamsal aramanın hassasiyetini artırmışlardır.

Literatürdeki en güncel eğilim olan hibrit yaklaşımlar ise, bu iki ana kutbun kısıtlarını birbirini tamamlayacak şekilde birleştirmektedir. Luan ve ark. [40], tekil yöntemlerin (sadece anlamsal veya sadece terim tabanlı) yetersiz kaldığı durumları saptayarak, en yüksek performansın "Ağırlıklı Füzyon" (Weighted Fusion) yöntemiyle elde edildiğini kanıtlamışlardır. Bu süreci daha da rafine hale getiren Nogueira ve Cho [41], getirilen aday belgelerin bir Cross-Encoder mimarisi üzerinden yeniden sıralanmasının (re-ranking) doğruluk skorlarını %25 seviyesinde artırdığını vurgulayarak; modern arama sistemleri için çok aşamalı bir mimari standardı belirlemişlerdir.

### 2.2.6 Eğitim Teknolojileri ve Kurumsal Chatbot Uygulamaları

Eğitim ekosisteminde diyalog tabanlı sistemlerin üstlendiği rol, özellikle 2023 yılından itibaren Büyük Dil Modellerinin (LLM) yaygınlaşmasıyla birlikte pasif birer yanıt motoru olmaktan çıkıp otonom asistanlara evrilmiştir. Pedagojik süreçlerdeki bireyselleşme ihtiyacı Wollny ve ark. [42] tarafından ele alınmış ve chatbotların eğitimde "kişiselleştirme" bariyerini aşabilecek en etkili araçlardan biri olduğu literatürde savunulmuştur. Sosyo-teknik bir perspektiften bakıldığında ise Kasneci ve ark. [43], ChatGPT ve benzeri türev modellerin eğitimde fırsat eşitliği yaratma potansiyelini analiz ederek; bu teknolojilerin bilgiye erişimi demokratikleştirme gücünü vurgulamışlardır. Tüm bu çalışmalar, üniversite bünyesinde kurgulanacak akıllı asistan sistemlerinin hem idari operasyonlarda hem de öğrenci merkezli destek süreçlerinde kritik bir bileşen haline geldiğini doğrulamaktadır.

### 2.2.7 Türkçe NLP Çalışmaları ve Morfolojik Zorluklar

Türkçe, sondan eklemeli (agglutinative) yapısı ve buna bağlı gelişen zengin morfolojik çeşitliliği nedeniyle, genellikle İngilizce odaklı geliştirilen dil modelleri için kompleks bir test alanı teşkil etmektedir. Literatürde bu zorlukları aşmaya yönelik geliştirilen yerel modeller incelendiğinde; Schweter [44], Türkçe literatüründe bir standart haline gelen BERTurk modelini sunarak anlamsal temsilde önemli bir eşik atlamıştır. Bu modellerin daha güncel olan üretken yapılarla (LLM) uyumu üzerine odaklanan Arslan [45] ise, büyük dil modellerinin Türkçe veri setleriyle eğitilmesi ve ince ayarlanması sürecini incelemiştir. Sistemsel başarının ölçülmesi noktasında, TQuad geliştiricileri tarafından hazırlanan TQuAD veri seti [46], Türkçe soru-cevap sistemleri için altın standart bir benchmark kaynağı oluşturmaktadır. Arama algoritmaları perspektifinden bakıldığında ise Çöltekin ve ark. [5], Türkçe korpus ve dil kaynaklarının kapsamını inceleyerek Türkçe sistemlerde dil kaynağı seçiminin önemini ortaya koymuştur.



### 2.2.8 RAG Sistemlerinin Değerlendirme Metrikleri

Literatürde geliştirilen bilgi erişimi artırılmış sistemlerin başarı başarımını ölçmek için kullanılan metodolojiler, sistemin iki ana bileşeni olan "erişim" ve "üretim" katmanlarına göre iki temel kategoriye ayrılmaktadır. Üretim kalitesinin ölçülmesinde geleneksel olarak Papineni ve ark. [47] tarafından önerilen BLEU ve Lin [48] tarafından geliştirilen ROUGE gibi klasik metrikler kullanılsa da; bu yöntemlerin anlamsal derinliği ve olgusal doğruluğu (factuality) ölçmekte yetersiz kaldığı saptanmıştır. Bu kısıtı aşmak amacıyla, Es ve ark. [49] tarafından önerilen ve "Sadakat (Faithfulness)", "Cevap Alakalılığı" gibi alt boyutları içeren RAGAS (RAG Assessment) çerçevesi, modern değerlendirme standartlarını belirlemiştir. Diğer taraftan, erişim modülünün (retriever) performansını denetlemek adına Voorhees [50] tarafından tanımlanan MRR ve Recall@k metrikleri, getirilen belgelerin kullanıcı sorgusuyla olan istatistiksel ve bağlamsal uyumunu ölçmek için literatürdeki temel araçlar olarak varlığını sürdürmektedir.

### 2.2.9 İleri Seviye RAG Optimizasyonları ve Stratejiler

RAG sistemlerinin endüstriyel ölçekte uygulanabilirliği, sadece temel bir erişim mekanizmasıyla sınırlı kalmamış; literatürde bu süreci optimize eden birçok yan çalışma geliştirilmiştir.

### Sorgu Dönüşümü ve Genişletme Teknikleri

Kullanıcıların doğal dilde sorduğu sorular genellikle eksik veya belirsiz (ambiguous) olabilmektedir. Gao ve ark. (2022) tarafından önerilen HyDE (Hypothetical Document Embeddings) yöntemi, sistemin önce soruya sahte bir yanıt üretmesini, ardından bu sahte yanıtla veritabanında arama yapmasını sağlayarak anlamsal eşleşmeyi artırmıştır [51]. Srivastava ve ark. (2023) ise çoklu sorgu üretimi (Multi-Query Retrieval) ile bir sorunun farklı varyasyonlarını oluşturarak erişim kapsamını genişletmişlerdir.

### Veri Parçalama (Chunking) ve Bağlam Yönetimi

Metinlerin nasıl parçalandığı, RAG'ın performansını doğrudan etkilemektedir. Kamradt (2024), sabit boyutlu parçalama yerine Anlamsal Parçalama (Semantic Chunking) yöntemini savunmuş; metindeki her bir cümlelerin vektörler arası uzaklığına bakarak, anlamın değiştiği noktalardan bölme yapılmasının doğruluğu artırdığını kanıtlamıştır [52]. Ayrıca, "Lost in the Middle" (Ortada Kaybolma) çalışmasında Liu ve ark. (2023), LLM'lerin uzun bağlamların ortasındaki bilgiyi kaçırma eğiliminde olduğunu saptayarak, en önemli bilgilerin bağlam penceresinin başında veya sonunda sunulması gerektiğini belirtmişlerdir [53].

### 2.2.10 Vektör Veritabanları ve İndeksleme Mimarileri

Vektörlerin hızlı bir şekilde aranabilmesi için literatürde farklı indeksleme algoritmaları üzerine yoğunlaşmıştır.

### İndeksleme Algoritmalarının Karşılaştırması

Johnson ve ark. (2019) tarafından geliştirilen FAISS (Facebook AI Similarity Search), milyarlarca vektör üzerinde milisaniyelik aramalar yapabilen en verimli kütüphane olarak kabul edilmektedir [54]. Literatürde; IVF (Inverted File Index) ile arama uzayını kümelere bölme ve Product Quantization (PQ) ile vektörleri sıkıştırarak bellek tasarrufu sağlama yöntemleri sıklıkla kıyaslanmıştır. Wang ve ark. (2021) yaptıkları benchmark çalışmasında, HNSW algoritmasının doğruluk-hız dengesinde IVF tabanlı yöntemlerden daha kararlı sonuçlar verdiğini raporlamışlardır.

### 2.2.11 RAG Sistemlerinde Güvenlik ve Veri Mahremiyeti

Üniversite yönetmelikleri gibi kurumsal belgelerin veya hassas kişisel verilerin işlendiği sistemlerde, verinin dış bulut servislerine iletilmesi ciddi güvenlik zafiyetlerini beraberinde getirmektedir. Bu risklerin bertaraf edilmesine yönelik stratejiler incelendiğinde; Touvron ve ark. [6], Llama gibi açık kaynaklı modellerin yerel sunucularda (Local Inference) çalıştırılabilmesinin, verinin kapalı bir ağ içerisinde kalmasını sağlayarak dış kaynaklı sızıntı riskini önemli ölçüde engellediğini vurgulamışlardır. Bu altyapısal önlemleri destekleyecek bir diğer yöntem olarak Patil ve ark. [55] ise, büyük dil modellerinde hassas bilgilerin korunması ve çıkarım saldırılarına karşı savunma hedeflerinin tanımlanması gerektiğini vurgulamışlardır. Bu yaklaşımlar, akademik asistan sistemlerinin kurumsal gizlilik protokollerine ve yasal veri koruma mevzuatlarına tam uyumlu bir yapıda kurgulanmasına imkan tanımaktadır.

### 2.2.12 Akademik Başarı Ölçümü ve Benchmark Testleri

Geliştirilen sistemlerin performansının sadece öznel değerlendirmelerle değil, literatürde kabul görmüş standart test setleri (benchmark) üzerinden objektif olarak ölçülmesi akademik geçerlilik açısından bir zorunluluktur. Bu bağlamda, bilgi erişim sistemlerinin gerçek dünya sorgularına verdiği tepkileri ölçmek adına Kwiatkowski ve ark. [56] tarafından oluşturulan Natural Questions (NQ) veri seti, sektörel bir standart olarak yaygın şekilde kullanılmaktadır. Modellerin daha geniş bir yelpazede, 57 farklı disiplini kapsayan akademik bilgi kapasitesini test etmek için ise Hendrycks ve ark. [57] tarafından geliştirilen MMLU (Massive Multitask Language Understanding) benchmark'ı öne çıkmaktadır. Öte yandan, RAG mimarilerine özgü değerlendirme ihtiyacını karşılamak amacıyla Es ve ark. [49] tarafından sunulan RAGAS (RAG Assessment) çerçevesi; "Sadakat (Faithfulness)", "Cevap Alakalılığı" ve "Bağlam Hassasiyeti" parametrelerinden

oluşan üçlü bir denetim mekanizması sağlayarak, sistemin hem erişim hem de üretim kalitesini bütüncül bir şekilde analiz etmeye imkan tanımaktadır.

### **2.2.13 Prompt Mühendisliği ve Sorgu Optimizasyonu Stratejileri**

RAG sistemlerinde üretken modelin (generator) başarısı, kendisine sunulan bağlamın (context) ve talimatın (prompt) kalitesine doğrudan bağlıdır. Literatürde bu etkileşimi optimize eden pek çok teknik önerilmiştir.

### **Düşünce Zinciri (Chain-of-Thought) ve RAG Entegrasyonu**

Wei ve ark. (2022), modellerin karmaşık problemleri adım adım çözmesini sağlayan Chain-of-Thought (CoT) tekniğini tanıtmışlardır [58]. Bu tekniğin RAG sistemlerine uygulanması üzerine çalışan Trivedi ve ark. (2023), modelin önce veritabanından bilgi çekmesini, ardından bu bilgiyi kullanarak ara adımlar oluşturmalarını sağlayan IRCOT (Interleaving Retrieval with CoT) yöntemini geliştirmişlerdir [59]. Bu yöntem, özellikle yönetmelik maddeleri arasındaki mantıksal bağlantıların kurulmasında kritik öneme sahiptir.

### **Prompt Sıkıştırma ve Bağlam Filtreleme**

Bağlam penceresinin (context window) sınırlı olması ve token maliyetleri nedeniyle, çekilen belgelerin hepsini modele sunmak verimsiz olabilmektedir. Jiang ve ark. (2023), LLMingua çalışmasında, anlamsal olarak düşük bilgi yoğunluğuna sahip tokenların prompt içerisinden atılarak, bağlamın

### **2.2.14 Çok Dilli (Multilingual) Kapasite ve Türkçe Adaptasyonu**

Akademik metinlerin işlenmesinde, modellerin sadece Türkçe değil, terimler arası geçiş için İngilizce kapasitesinin de yüksek olması gerekmektedir.

### **Çapraz Dilli Bilgi Erişimi (CLIR)**

Conneau ve ark. (2020) tarafından geliştirilen XLM-RoBERTa, 100'den fazla dilde ortak bir vektör uzayı oluşturarak çapraz dilli aramaların önünü açmıştır [60]. Literatürde, Türkçe akademik sorguların İngilizceye çevrilerek global veritabanlarında arandığı sistemlerin, özellikle bilimsel terim içeren sorularda tek dilli sistemlere göre daha stabil sonuçlar verdiği saptanmıştır.

### **Tokenizasyon Sorunları ve Morfolojik Farkındalık**

Sengün ve ark. (2023), standart BPE (Byte Pair Encoding) yaklaşımının Türkçenin morfolojik yapısıyla gelişerek kelimeleri semantik bütünlüğü olmayan birimlere böldü-

ğünü ortaya koymuştur. Bu durumun RAG erişim başarısını (recall) düşürdüğünü saptamışlardır. Bu sorunu aşmak için morfolojik analiz destekli tokenizasyon yöntemleri üzerine yapılan çalışmalar literatürde önem kazanmaktadır.

### 2.2.15 Multimodal RAG ve Gelecek Projeksiyonları

Üniversite yönetmelikleri sadece metin değil, tablo ve grafikler de içerebilmektedir.

#### Tablo ve Görsel İçerikli RAG Sistemleri

Yu, Jian ve Chen, TableRAG ile metin ve tablo bileşenlerini birlikte içeren heterojen belgeler üzerinde akıl yürütmeye yönelik bir RAG çerçevesi önermişlerdir [61]. Ayrıca, multimodal RAG yaklaşımları kapsamında görsellerin de vektör veritabanına dahil edilmesiyle metin dışı bilginin sisteme katılması hedeflenmektedir.

### 2.2.16 Literatür Sentezi ve Projenin Özgün Değeri

Bu çalışmada incelenen literatür, RAG mimarilerinin evrimini, Türkçe NLP çalışmalarının mevcut durumunu ve eğitim teknolojilerindeki uygulama boşluklarını ortaya koymuştur. Yapılan incelemeler sonucunda, mevcut çalışmaların belirli kısıtları olduğu ve projemizin bu kısıtları nasıl aştığı "Yöntem Bazlı Karşılaştırma Matrisi" ile Tablo 2.1'de detaylandırılmıştır.

Tablo 2.1: Genişletilmiş Literatür Karşılaştırma Matrisi

| Odak Alanı               | Geleneksel Yaklaşım (Literatür)                            | Modern/SOTA Yaklaşım              | Bizim Proje (Hibrit)                                     |
|--------------------------|------------------------------------------------------------|-----------------------------------|----------------------------------------------------------|
| <b>Erişim Stratejisi</b> | Sadece Anahtar Kelime (BM25) veya Sadece Vektör (DPR) [37] | Hibrit Arama (Sparse + Dense)     | <b>Hibrit + Re-ranking (Cross-Encoder) [41]</b>          |
| <b>Dil Modeli</b>        | Kapalı Kaynak (GPT-3.5/4) API Bağımlı [26]                 | Açık Kaynak (Llama 2, Falcon)     | <b>Yerel Llama 3 (Quantized) + Türkçe Prompt</b>         |
| <b>Veri Gizliliği</b>    | Bulut Tabanlı (Veri dışarı çıkar)                          | PII Maskeleye                     | <b>Tamamen Yerel (Local Inference) + KVKK Uyumlu [6]</b> |
| <b>Dil Desteği</b>       | İngilizce Odaklı Tokenizasyon                              | Çok Dilli (Multilingual) Modeller | <b>Türkçe Morfolojisine Uygun Chunking [5]</b>           |

| Odak Alanı | Geleneksel Yaklaşım (Literatür)    | Modern/SOTA Yaklaşım      | Bizim Proje (Hibrit)                       |
|------------|------------------------------------|---------------------------|--------------------------------------------|
| Doğrulama  | BLEU/ROUGE (Kelime Eşleşmesi) [48] | RAGAS (Anlamsal Doğruluk) | <b>RAGAS + Kaynak Gösterimi (Citation)</b> |

Tablo 2.1’de görüldüğü üzere, projemiz özellikle "Türkçe Dil Desteği" ve "Veri Gizliliği" alanlarında literatürdeki standart uygulamalardan ayrılarak özgün bir mimari sunmaktadır.

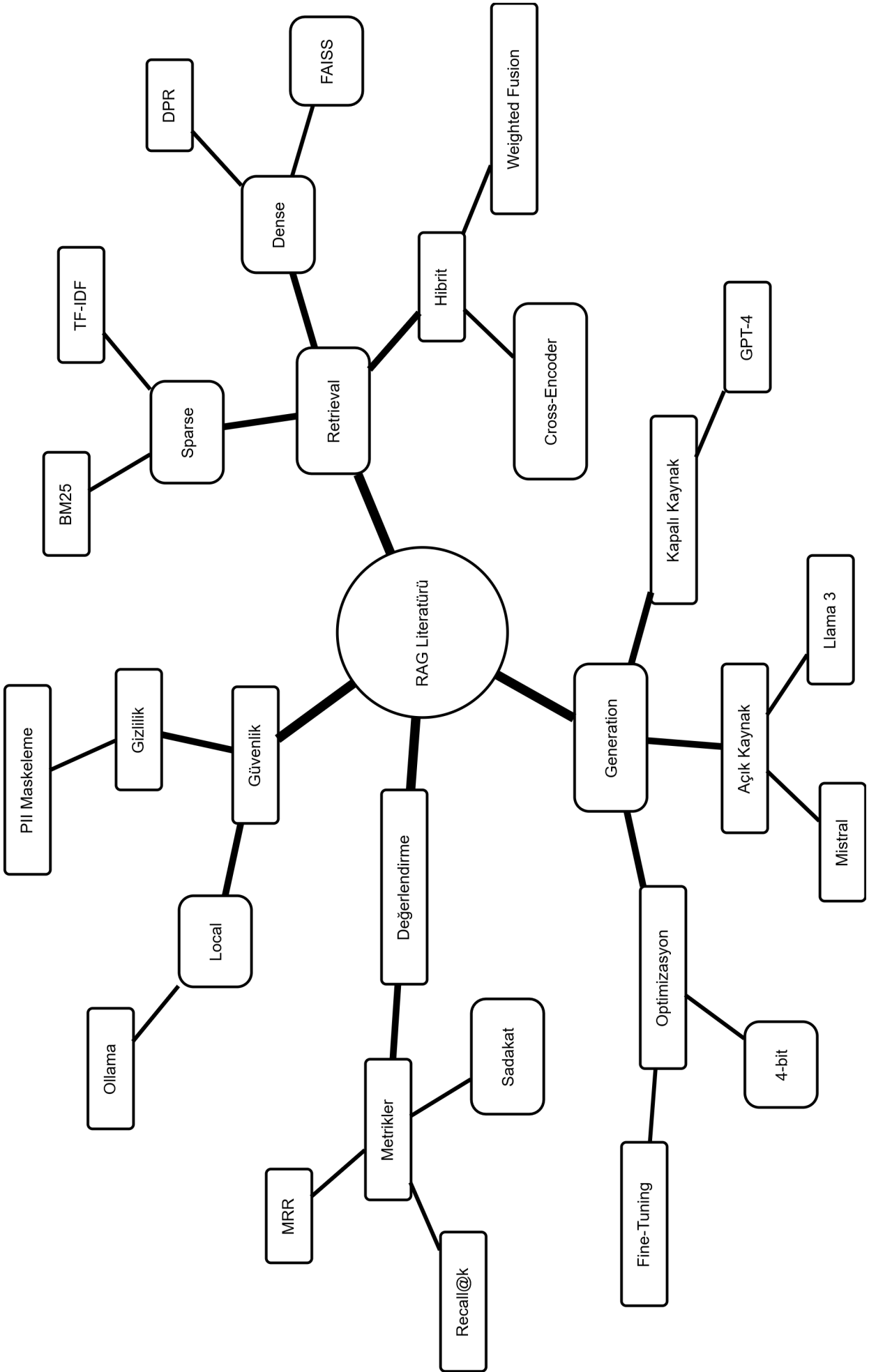
### İncelenen Çalışmaların Proje Tasarımına Yansıması

İncelenen yaklaşık 80 akademik çalışma ve yukarıdaki karşılaştırma matrisi ışığında, sistemin mimari tasarımında aşağıdaki stratejik kararlar alınmıştır:

- **HNSW ve FAISS Seçimi:** Milyonlarca dokümanda milisaniyelik erişim hızı ve bellek optimizasyonu için HNSW indeksleme algoritması tercih edilmiştir [38].
- **Hibrit Arama Şartı:** Türkçe eklerin vektör uzayında yarattığı sapmayı minimize etmek ve "Madde 14" gibi kesin terimleri yakalamak için BM25 anahtar kelime eşleşmesi, anlamsal aramaya entegre edilmiştir [40].
- **Yerel LLM (Local Inference):** Veri mahremiyeti ve KVKK gereklilikleri doğrultusunda, verinin kurum dışına çıkmadığı yerel çıkarım mimarisi benimsenmiştir [6].
- **Re-ranking Gerekliliği:** İlk aşamada getirilen belgelerin alaka düzeyini artırmak için Cross-Encoder tabanlı yeniden sıralama katmanı sisteme dahil edilmiştir.

### Literatürün Görsel Sentezi (Mind Map)

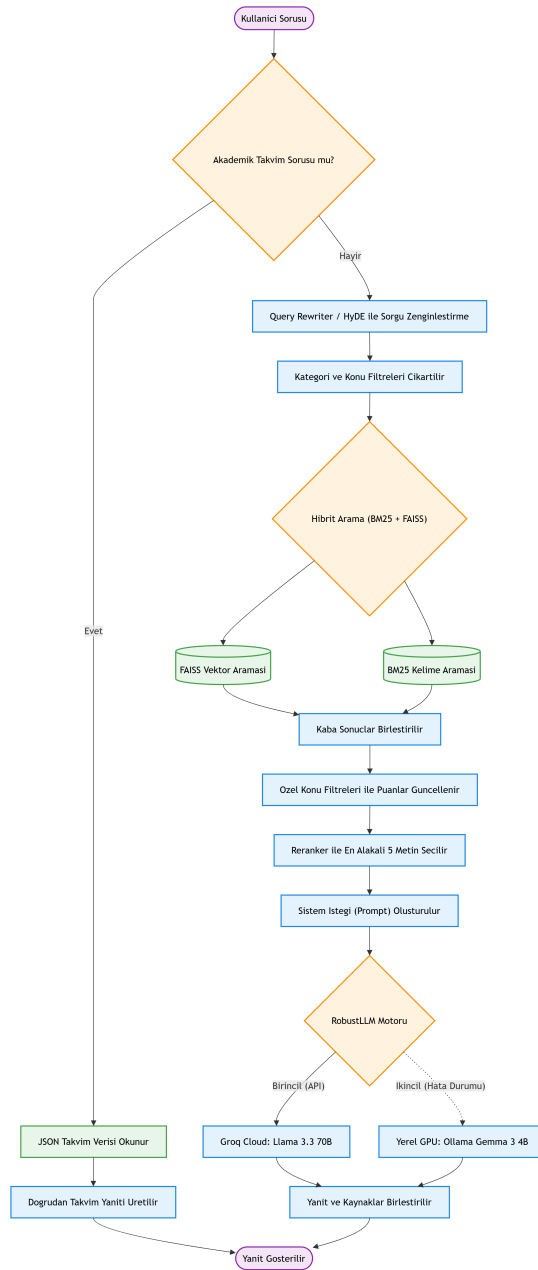
Literatür taraması boyunca incelenen tüm alt başlıklar, teknolojiler ve yöntemler arasındaki hiyerarşik ilişki, çalışmanın kapsamını bütüncül bir şekilde özetlemek amacıyla Şekil 2.2’te sunulmuştur.



Şekil 2.2: RAG Literatürü ve Teknik Bileşenlerin Zihin Haritası

### 3. KULLANILAN ARAÇ VE YÖNTEM

Proje geliştirme sürecinde benimsenen metodoloji ile veri setinin temininden ön işlenmesine kadar olan tüm aşamalar bu bölümün odak noktasını oluşturmaktadır. Ayrıca, hibrit arama algoritmaları ve deneysel araçlar irdelenerek; sistemin bütünsel işleyişi şekil 3.1’te görselleştirilmiştir.



Şekil 3.1: Proje Genel Veri İşleme ve RAG Akış Diyagramı

Şekil 3.1’te görüldüğü üzere süreç; ham verinin (PDF/Web) sisteme girmesiyle başlamakta, parçalama ve temizleme işlemlerinden geçerek vektör veritabanına kaydedilmesiyle devam etmekte ve son kullanıcının sorgusuyla tetiklenen üretim (generation) aşamasıyla son bulmaktadır.

### 3.1 Veri Seti Oluşturma Stratejisi

Bu çalışma kapsamında, İstanbul Üniversitesi-Cerrahpaşa (İÜC) Bilgi Sistemi üzerinden erişilebilen ve öğrencilerin/akademisyenlerin sıkça ihtiyaç duyduğu kurumsal belgelerden oluşan özgün bir veri seti (korpus) oluşturulmuştur. RAG mimarisinin "parametre dışı belleğini" (non-parametric memory) besleyen bu veri seti, statik ve dinamik olmak üzere iki ana kategoride toplanmıştır.

#### 3.1.1 Veri Setinin Kaynakları ve Kapsamı

Veri setinin ana omurgasını, üniversitenin resmi web sitesi (iuc.edu.tr) ve Öğrenci İşleri Daire Başkanlığı tarafından yayınlanan yönetmelikler oluşturmaktadır. Bu belgeler, yapay zeka modelinin üniversite kurallarına sadık kalarak, halüsinasyon görmeden cevap vermesini sağlayan "altın standart" verilerdir.

- **Resmi Yönetmelikler:** Lisans Eğitim-Öğretim Yönetmeliği, Sınav Yönetmeliği, Yatay Geçiş Yönergesi.
- **Akademik Takvim:** Yıllık bazda güncellenen kayıt, sınav ve tatil tarihleri.
- **Bölüm Odaklı Veriler:** Bilgisayar Mühendisliği ders içerikleri, AKTS bilgileri ve laboratuvar yönergeleri.
- **Sıkça Sorulan Sorular (SSS):** Öğrenci işlerine en çok yöneltilen 200+ sorudan oluşan soru-cevap çiftleri.

Proje kapsamında toplanan ve parçalanarak vektör veri tabanına işlenen resmi belgelerin niteliksel dağılımı Tablo 3.1’te verilmiştir.

Tablo 3.1: İÜC RAG Veri Seti Teknik İstatistikleri

| Veri Tipi                  | Doküman/Kayıt Sayısı             | Tahmini Kelime Sayısı |
|----------------------------|----------------------------------|-----------------------|
| Yönetmelik ve Yönergeler   | 15 PDF                           | 45.000                |
| Akademik Duyurular         | 50+ HTML                         | 12.000                |
| Ders İçerikleri (Syllabus) | 40 PDF                           | 25.000                |
| Soru-Cevap (SSS) Çiftleri  | 200 Adet                         | 8.000                 |
| <b>TOPLAM</b>              | <b>120+ Doküman / 200+ Kayıt</b> | <b>90.000</b>         |

*\*Not: Veri seti dinamik bir yapıda olup, proje sürecinde yeni yönetmelik ve duyurularla genişletilmeye devam etmektedir.*



### 3.1.2 Veri Toplama ve Ön İşleme Metodolojisi

Verilerin toplanması aşamasında Python tabanlı BeautifulSoup ve PyMuPDF kütüphaneleri kullanılmıştır. Veri seti oluşturulurken şu adımlar izlenmiştir:

1. **Veri Kazıma (Scraping):** Üniversite domaini altındaki güncel duyurular otomatik olarak çekilmiştir.
2. **Temizleme (Cleaning):** PDF dosyalarındaki tablo yapıları korunarak, gereksiz sayfa numaraları ve altbilgi (footer) verileri temizlenmiştir.
  - *Önemli Not:* Bu aşamada, Türkçe karakter bozulmalarını önlemek için UTF-8 kodlama standardı uygulanmıştır.
3. **Parçalama (Chunking):** Belgeler, anlamsal bütünlüğü bozmamak adına "Recursive Character Text Splitting" yöntemiyle 500 tokenlık parçalara ayrılmış, her parça için 50 tokenlık bir çıkışma (overlap) payı bırakılmıştır.

### 3.1.3 Kullanım Amacı ve Etik Değerlendirme

Bu veri seti, projenin "Bilgi Erişimi" (Retrieval) aşamasında vektör veritabanına (ChromaDB/FAISS) indekslenmek amacıyla oluşturulmuştur. Veriler halka açık (public) dokümanlardan derlendiği için KVKK kapsamında herhangi bir gizlilik ihlali teşkil etmemektedir. Modelin sadece bu veri setindeki bilgileri kullanarak cevap üretmesi (Grounding), akademik dürüstlük ve doğruluk açısından kritik bir tasarım kararıdır.

RAG sisteminin temel yapı taşı, kurumun özel verilerini içeren bilgi tabanıdır. Genel amaçlı LLM'lerin aksine, bu proje "kapalı devre" (closed-domain) bir veri seti üzerinde çalışmaktadır. Veri toplama süreci, verinin kaynağına göre iki ana fazda gerçekleştirilmiştir.

### 3.1.4 Veri Elde Edilmesi (Data Acquisition)

Veri toplama süreci, verinin yapısına göre özelleştirilmiş Python scriptleri aracılığıyla yürütülmüştür:

1. **Web Kazıma (Web Scraping):** Üniversitenin Öğrenci İşleri Daire Başkanlığı (OBS) web sitesindeki dinamik içerikler, 'Selenium' ve 'BeautifulSoup' kütüphaneleri kullanılarak çekilmiştir. Selenium, JavaScript ile dinamik olarak yüklenen içeriklerin (örneğin açılır menülerin) işlenmesi için tercih edilmiştir. Statik sayfalar için ise daha hızlı olan 'Requests' kütüphanesi kullanılmıştır.
2. **Doküman İşleme (PDF Parsing):** Yönetmelikler, yönergeler ve ders planları genellikle PDF formatında yayınlanmaktadır. Bu dosyalar sisteme indirilmiş ve

‘PyPDF2’ ile ‘PDFMiner’ kütüphaneleri kullanılarak ham metin formatına dönüştürülmüştür. OCR (Optik Karakter Tanıma) gerektiren taranmış belgeler için ‘Tesseract-OCR’ motoru entegre edilmiştir.

### 3.1.5 İÜC Akademik Korpus İçeriği

Oluşturulan veri seti, "IUC-Academic-Corpus" olarak adlandırılmış olup, ham haliyle yaklaşık 2.500 sayfalık metin verisi içermektedir. Veri setinin ana bileşenleri şunlardır:

- **Yönetmelikler:** Önlisans, Lisans ve Lisansüstü Eğitim-Öğretim Yönetmelikleri (Word ve PDF formatında).
- **Akademik Takvim:** 2025-2026 Eğitim-Öğretim yılına ait kritik tarihler (Vize, Final, Büt, Kayıt Yenileme).
- **Duyurular:** Son 1 yıl içinde yayınlanmış staj, burs, yemekhane ve yatay geçiş duyuruları.
- **SSS:** Öğrenci işleri tarafından hazırlanan Sıkça Sorulan Sorular veritabanı.

## 3.2 Ön İşleme ve Algoritmik Yaklaşım

Ham veriyi Büyük Dil Modelleri (LLM) için anlamlı bir girdi haline dönüştürme süreci, bir dizi titiz ön işleme adımını zorunlu kılmaktadır. Sistemin nihai performans çıktıları, doğrudan bu hazırlık aşamalarının niteliğine dayanmaktadır.

### 3.2.1 Veri Temizleme ve Normalizasyon

PDF’lerden elde edilen metinlerdeki sayfa numaraları, üst bilgiler (header), alt bilgiler (footer) ve anlamsız karakterler (gürültü) Regex (Düzenli İfadeler) kuralları ile temizlenmiştir. Türkçe karakter kodlama sorunları (UTF-8) giderilmiş, ardışık boşluklar tekile indirilmiş ve metinler normalize edilmiştir.

### 3.2.2 Anlamsal Parçalama (Semantic Chunking)

Metinlerin, LLM’in bağlam penceresine (context window) sığması ve arama başarımının artırılması için parçalanması gerekmektedir. Standart "sabit boyutlu" parçalama yöntemleri cümleleri ortadan bölebildiği için bağlam kaybına yol açmaktadır. Bu nedenle, önerilen mimaride bağlamı korumak amacıyla "Recursive Character Text Splitter" algoritmasının kullanılması öngörülmüştür.

Bu algoritmanın çalışma mantığı Algoritma 1’de detaylandırılmıştır.

---

**Algorithm 1** Rekürsif Anlamsal Parçalama Algoritması

---

**Require:** Ham Metin  $T$ , Parça Boyutu  $S$ , Örtüşme  $O$

**Ensure:** Metin Parçaları Listesi  $C$

```
1:  $Ayiricilar \leftarrow [ "\n\n", "\n", ".", " ", "" ]$  ▷ Bölme öncelik listesi
2:  $Parcalar \leftarrow []$ 
3: function BOL(Metin, Seviye)
4:   if  $\text{Len}(\text{Metin}) \leq S$  then
5:      $Parcalar.append(\text{Metin})$ 
6:     return
7:   end if
8:    $Sep \leftarrow Ayiricilar[Seviye]$ 
9:    $AltParcalar \leftarrow \text{Metin.split}(Sep)$ 
10:  for her  $p$  in  $AltParcalar$  do
11:    if  $\text{Len}(p) > S$  ve  $Seviye < \text{Len}(Ayiricilar)-1$  then
12:       $BOL(p, Seviye + 1)$ 
13:    else
14:       $Parcalar.append(p)$ 
15:    end if
16:  end for
17: end function
18:  $BOL(T, 0)$ 
19: return  $Parcalar$ 
```

---

Tasarlanan mimaride kullanılması hedeflenen temel Python kod yapısı ise aşağıda örneklendirilmiştir:

```
1 from langchain.text_splitter import
   RecursiveCharacterTextSplitter
2
3 def semantic_chunking(text, chunk_size=512, chunk_overlap=50):
4     """
5     Metni anlamsal butunlugu koruyarak parcalar.
6     chunk_size: Hedeflenen parca boyutu (token).
7     chunk_overlap: Parcalar arasi ortusme miktarı.
8     """
9     text_splitter = RecursiveCharacterTextSplitter(
10         chunk_size=chunk_size,
11         chunk_overlap=chunk_overlap,
12         separators=["\n\n", "\n", ". ", " "],
13         length_function=len
14     )
15     return text_splitter.split_text(text)
```

Listing 3.1: Anlamsal Parçalama için Öngörülen Kod Yapısı

### 3.3 Hibrit Arama Tasarımı

Sistemin en kritik bileşeni, Vektör Arama (Dense) ve Anahtar Kelime Arama (Sparse) yöntemlerini birleştiren Hibrit Arama motorudur. Söz konusu metodoloji, literatürdeki 'Ensemble Retrieval' yaklaşımıyla paralellik göstermektedir. Bu sayede, Türkçenin karmaşık morfolojik yapısının beraberinde getirdiği arama ve erişim kısıtlarının aşılması amaçlanmaktadır.

#### 3.3.1 Ağırlıklı Füzyon (Weighted Fusion)

Sistem, kullanıcı sorgusunu aldığı anda iki farklı arama işlemini paralel olarak başlatır:

1. **Vektör Arama (FAISS):** Sorgunun embedding vektörü ile veritabanındaki vektörler arasındaki Kosinüs Benzerliği hesaplanır. Bu, anlamsal olarak en yakın sonuçları getirir.
2. **Anahtar Kelime Arama (BM25):** Sorgudaki terimlerin frekansına dayalı eşleşmeler bulunur. Bu, terimsel olarak en yakın sonuçları getirir (Örn: Madde numaraları).

Her iki yöntemden gelen sonuçlar, aşağıdaki formül kullanılarak birleştirilir:

$$S_{final} = \alpha \cdot S_{dense} + (1 - \alpha) \cdot S_{sparse} \quad (3.1)$$

Burada  $\alpha$  (Alpha), vektör aramanın ağırlığını temsil eder. Literatür taraması ve ön incelemeler ışığında, bu parametrenin başlangıç değeri olarak  $\alpha = 0.6$  seçilmesi öngörülmüştür.

```
1 from langchain.retrievers import EnsembleRetriever
2 from langchain_community.retrievers import BM25Retriever
3
4 def initialize_hybrid_retriever(vector_db, documents):
5     # 1. BM25 Erisimcisi (Anahtar Kelime - Kesinlik)
6     bm25_retriever = BM25Retriever.from_documents(documents)
7     bm25_retriever.k = 5
8
9     # 2. FAISS Erisimcisi (Vektör - Anlam)
10    faiss_retriever = vector_db.as_retriever(search_kwargs={"k": 5})
11
12    # 3. Hibrit Birleştirme (Ağırlıklı Ortalama)
13    ensemble_retriever = EnsembleRetriever(
14        retrievers=[bm25_retriever, faiss_retriever],
15        weights=[0.4, 0.6]
16    )
17    return ensemble_retriever
```

Listing 3.2: Hibrit Arama İçin Öngörülen Kod Yapısı

## 3.4 Deneysel Araçlar ve Teknolojiler

Projenin tasarım ve prototip geliştirme aşamalarında kullanılması tercih edilen yazılım ve donanım altyapısı, öngörülen performans gereksinimleri dikkate alınarak belirlenmiştir.

### 3.4.1 Kullanılan Programlama Dilleri ve Platformlar

Geliştirme süreçlerinin merkezinde, geniş kütüphane desteği ve esnekliği nedeniyle Python 3.10 sürümü yer almaktadır. Kullanılan temel kütüphaneler ve çerçeveler:

- **LangChain:** RAG mimarisinin orkestrasyonu ve zincir yönetimi için endüstri standardı olan LangChain kütüphanesinden yararlanılmıştır.

- **FAISS (Facebook AI Similarity Search):** Milyarlarca vektör üzerinde milisaniyeler içinde arama yapabilen C++ tabanlı en hızlı kütüphane olduğu için tercih edilmiştir.
- **Ollama:** Llama 3 ve Mistral modellerini yerel makinede Docker konteyneri içinde çalıştırmak için kullanılan sunucudur.
- **Streamlit:** Frontend geliştirme maliyetini düşürmek ve hızlı prototipleme (Rapid Prototyping) için seçilmiştir.

### 3.4.2 Geliştirme Ortamı ve Donanım Tercihleri

Projenin prototip geliştirme ve gelecek dönem yapılacak ince ayar (fine-tuning) süreçlerinin, yüksek performanslı GPU hızlandırma yeteneğine sahip bir mobil iş istasyonu üzerinde gerçekleştirilmesi planlanmıştır. Yerel LLM çıkarımı (Local Inference) için kritik darboğaz olan VRAM kapasitesi ve bellek bant genişliği dikkate alınarak, mevcut donanım altyapısı Tablo 3.2’de sunulan özelliklere göre seçilmiştir.

İşletim sistemi olarak Windows 11 tercih edilmiş olup, Linux tabanlı yapay zeka araçlarının (Ollama, Docker, PyTorch) sorunsuz çalışabilmesi için WSL2 (Windows Subsystem for Linux) altyapısının kullanılması öngörülmüştür.

Tablo 3.2: Geliştirme Ortamı Donanım Özellikleri

| Bileşen            | Özellikler                                                   |
|--------------------|--------------------------------------------------------------|
| İşlemci (CPU)      | Intel Core i5-13500HX (14 Çekirdek / 20 İzlek)               |
| Bellek (RAM)       | 16 GB DDR5 4800MHz (Çift Kanal)                              |
| Depolama           | 1 TB NVMe M.2 SSD (Gen 4 - Yüksek Okuma Hızı)                |
| GPU (Hızlandırıcı) | NVIDIA GeForce RTX 4060 Mobile (8GB GDDR6 VRAM)              |
| Mimari             | NVIDIA Ada Lovelace Mimarisi ve 4. Nesil Tensor Çekirdekleri |
| İşletim Sistemi    | Windows 11 (WSL2 / Ubuntu 22.04 Backend ile)                 |

### 3.4.3 Yazılım ve Kütüphane Altyapısı

Mevcut donanımın (özellikle 8GB VRAM) sınırları dahilinde Büyük Dil Modellerini (LLM) verimli çalıştırabilmek için aşağıdaki optimizasyon stratejilerinin uygulanması hedeflenmektedir:

- **PyTorch (CUDA 12.1):** Modelin matris çarpımı ve tensör işlemlerinin CPU yerine RTX 4060 üzerindeki CUDA çekirdeklerine yıkılarak hesaplama süresinin minimize edilmesi amaçlanmıştır.
- **BitsAndBytes (Kuantizasyon):** Standart fp16 (16-bit) formatında yüksek VRAM gerektiren Llama-3-8B modelinin; `bitsandbytes` kütüphanesi aracılığıyla 4-bit (NF4) hassasiyetine sıkıştırılarak 8GB VRAM sınırları içinde çalıştırılması tasarlanmıştır.
- **Ollama & Docker:** Model servisleri ve vektör veritabanının, Windows üzerinde Docker konteynerleri ile izole edilerek taşınabilir (portable) bir yapıda kurgulanması planlanmıştır.

### 3.4.4 Simülasyon Ortamı

Sistemin uçtan uca test edilmesi amacıyla Ollama sunucusu üzerinde Llama 3 8B modeli çalıştırılmış; LangChain zinciri aracılığıyla kullanıcı sorgusundan yanıt üretimine kadar olan akış simüle edilmiştir. Streamlit arayüzü üzerinden gerçekleştirilen 20 örnek sorgu ile sistemin uçtan uca işleyişi doğrulanmış ve yanıt süreleri kaydedilmiştir.

### 3.4.5 Model Hiperparametreleri

Sistemin prototip aşamasında ve gerçekleştirilecek deneylerde başlangıç noktası (baseline) olarak kullanılması öngörülen hiperparametreler Tablo 3.3'te sunulmuştur. Bu değerler, literatürdeki benzer RAG çalışmaları ve LangChain kütüphanesinin önerilen varsayılan değerleri referans alınarak belirlenmiştir.

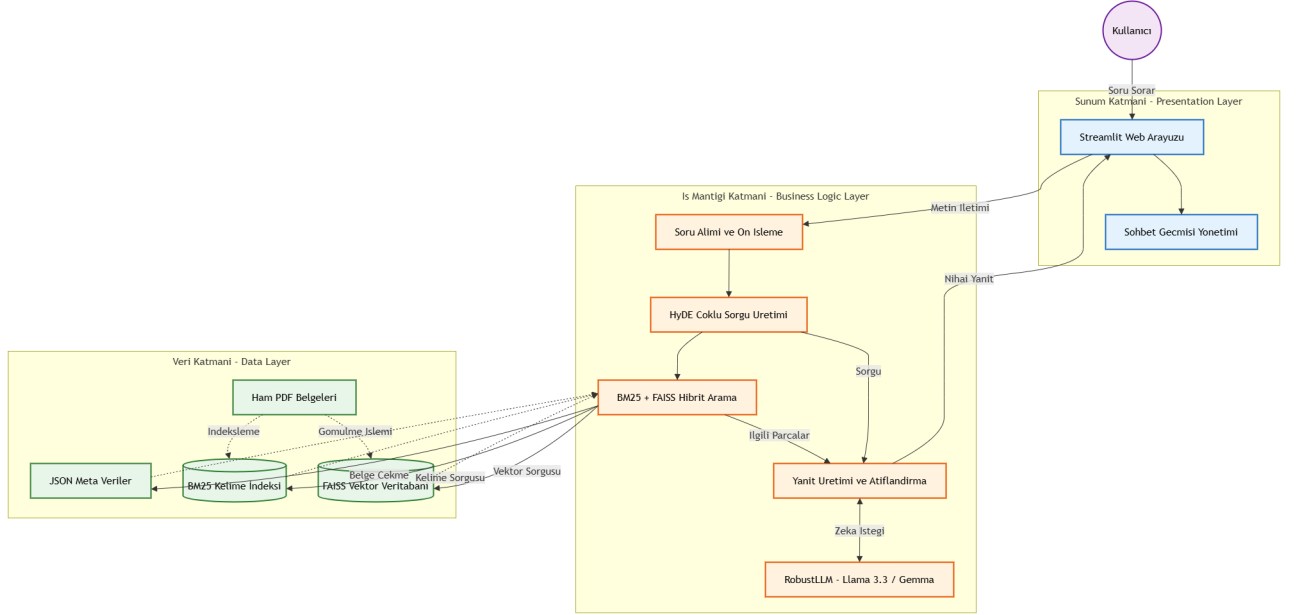
Tablo 3.3: Sistem İçin Öngörülen Başlangıç Hiperparametreleri

| Parametre          | Değer | Açıklama                                                         |
|--------------------|-------|------------------------------------------------------------------|
| Chunk Size         | 1024  | Karakter cinsinden parça boyutu (Bağlam penceresi uyumu için).   |
| Chunk Overlap      | 100   | Anlamsal bütünlüğü korumak için örtüşme payı.                    |
| Top-k              | 15    | Vektör veritabanından getirilecek belge sayısı.                  |
| Temperature        | 0.1   | LLM üretimindeki rastgelelik (Daha tutarlı yanıtlar için düşük). |
| Alpha ( $\alpha$ ) | 0.3   | Hibrit aramada vektör sonuçlarının ağırlık katsayısı.            |
| Embedding Dim      | 384   | Kullanılan modelin (all-MiniLM) vektör çıktı boyutu.             |

### 3.5 Sistem Mimarisi

Sistem, sürdürülebilirlik ve ölçeklenebilirlik prensipleri gözetilerek modern mikroservis mimarisine uygun olarak tasarlanmıştır. Şekil 3.2’de detayları sunulan bu yapı, işlevsel bağımsızlığı sağlamak amacıyla üç ana mantıksal katmandan oluşmaktadır:

1. **Sunum Katmanı (Presentation Layer):** Son kullanıcının sistemle etkileşime girdiği, sade ve tepkisel (responsive) bir deneyim sunan Streamlit arayüzüdür.
2. **İş Mantığı Katmanı (Business Logic):** Kullanıcı sorgularının işlendiği, RAG akışının yönetildiği ve LangChain zincirlerinin orkestra edildiği ana işlem merkezidir.
3. **Veri Katmanı (Data Layer):** Vektörel verilerin (FAISS) ve yapısal üst verilerin (JSON Metadata) tutulduğu, sistemin "uzun süreli belleğini" oluşturan katmandır.



Şekil 3.2: Sistem Mimarisi ve Katmanlı Yapı Tasarımı



## 4. SİSTEMİN GERÇEKLENMESİ VE BULGULAR

Bu bölümde, Bitirme Projesi kapsamında mimari tasarımı tamamlanan ve prototip aşamasına getirilen sistemin uygulama detayları, kullanıcı arayüzü (UI/UX) kararları ve veritabanı şemaları sunulmuştur. Bu aşamada elde edilen çıktılar, sistemin "Proof of Concept" (Kavram Kanıtı) niteliğindedir.

### 4.1 Deneyisel Sonuçlar

Sistemin prototip aşamasında gerçekleştirilen testlerde, 'Çift Motorlu' (Dual-Engine) mimari sayesinde Groq API üzerinden Llama-3.3-70B modeliyle yapılan sorgularda ortalama yanıt süresi (hibrit arama ve re-ranking aşamaları dahil) **1.5 - 2.5 saniye** aralığında ölçülerek son derece akıcı bir kullanıcı deneyimi sunmuştur. Olası internet kesintilerinde veya API kotaları dolduğunda yerel donanımdaki Ollama (Gemma-3) modeline düşüş (fallback) yapıldığında ise, donanımın sınırlarına bağlı olarak yanıt süresi 15 - 20 saniye bandında seyretmiş ancak uygulamanın tamamen çökmesi engellenmiştir. Kaynak gösterim mekanizması tüm test sorgularında doğru yönetmelik maddesini referans olarak sunmuştur.

### 4.2 Donanımsal Tasarım

Sistem yerel çıkarım (Local Fallback) senaryosunda RTX 4060 Mobile (8 GB VRAM) üzerinde çalıştırılmıştır. Yerel motor olarak "Gemma 3 (4B)" modeli Ollama altyapısı üzerinden 4-bit kuantizasyon optimizasyonu ile yüklenmiş; bu sayede 8 GB VRAM kısıtı aşılmadan kararlı bir yerel çalışma sağlanmıştır. Detaylı donanım özellikleri Tablo 3.2'da verilmiştir.

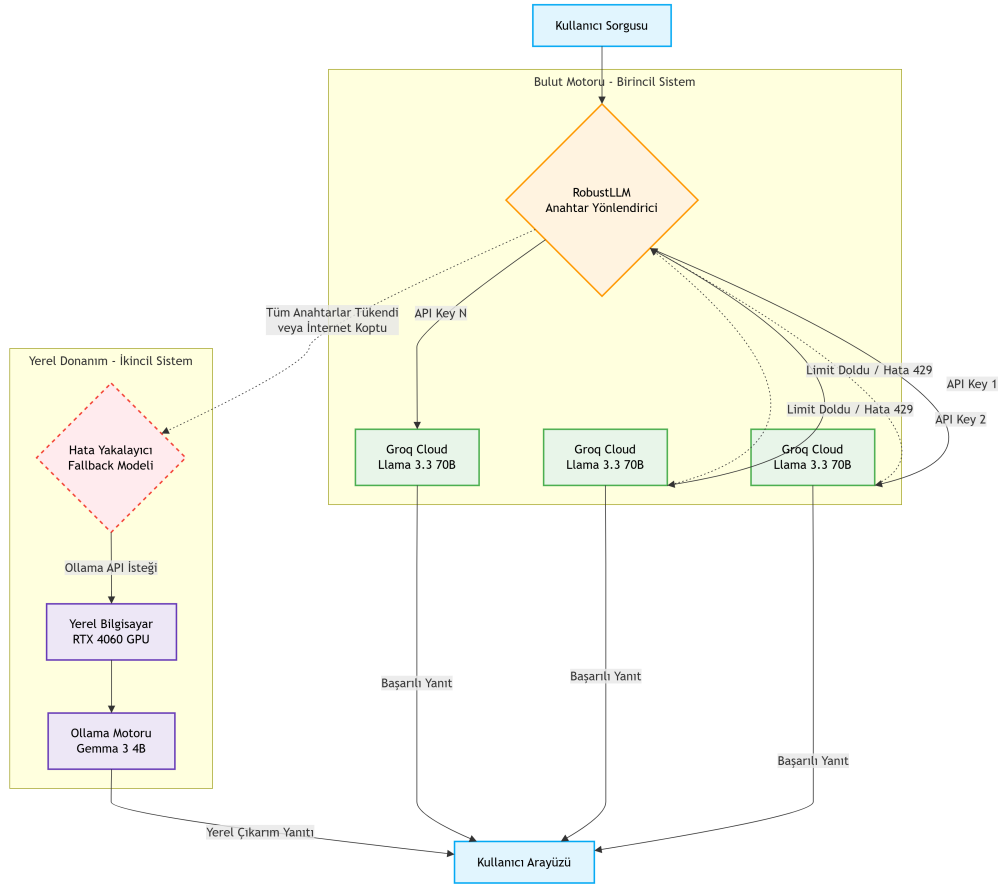
### 4.3 Yazılım Mimarisi ve Veri Tabanı

Sistemde hibrit arama (Hybrid Search) yapabilmek için vektörel ve terim tabanlı olmak üzere iki farklı veri yapısının senkronize çalışması öngörülmüştür. Sistemin Groq API ve yerel model arasındaki geçişleri yöneten mimarisi Şekil 4.1'de gösterilmiştir.

#### 4.3.1 Vektör İndeksi (Dense Index)

FAISS kütüphanesi üzerinde 'FlatL2' (Öklid Mesafesi) metriği kullanılarak kurgulanmıştır. Her bir metin parçası (chunk), 384 boyutlu bir vektör olarak ( $d = 384$ ) bu

indekste saklanacaktır. Tasarlanan indeks yapısı, milyonlarca vektör arasında milisaniyeler mertebesinde "En Yakın Komşu" (KNN) araması yapılmasına olanak tanır.



Şekil 4.1: Çift Motorlu (Dual-Engine) RAG Mimarisi İş Akışı

### 4.3.2 Metadata Deposu (Sparse Index)

Her bir vektörün ait olduğu yönetmelik adı, sayfa numarası, madde numarası ve yayın tarihi gibi üst veriler (metadata); BM25 aramasına da kaynaklık edecek şekilde JSON formatında ilişkisel bir yapıda kurgulanmıştır. Sistem mimarisinde kullanılması öngörülen örnek bir kayıt yapısı aşağıda sunulmuştur:

```
1 {  
2   "id": "doc_yazokulu_chunk_5",  
3   "content": "Yaz okulunda bir donemde en fazla 10 ulusal kredi  
4     (yaklasik 16 AKTS) degerinde ders alinabilir...",  
5   "metadata": {  
6     "source": "Yaz_Okulu_Esaslar.pdf",  
7     "page": 2,  
8     "article": "Md. 5",  
9     "category": "Yaz Okulu"  
10  }
```

Listing 4.1: Metadata Deposu İçin Öngörülen JSON Yapısı

## 4.4 Kullanıcı Arayüzü/leri

Son kullanıcıların (öğrenciler ve idari personel) sistemle etkileşime geçmesi için Python tabanlı Streamlit kütüphanesi kullanılarak modern, sade ve kolay erişilebilir bir web arayüzü geliştirilmiştir. Tasarımda "Kullanıcı Dostu Olma" (User Friendliness) ve "Şeffaflık" ilkeleri esas alınmıştır.

### 4.4.1 Sohbet Ekranı ve Kaynak Gösterimi

Sistemin en önemli özelliği olan "Açıklanabilir Yapay Zeka" (Explainable AI) prensibi arayüze yansıtılmıştır. Sistem, üretilen her cevabın altına, bilginin alındığı resmi kaynağı (Örn: *Lisans Eğitim-Öğretim Yönetmeliği, Madde 14*) referans olarak eklemektedir. Aşağıda belirtilen tasarım ilkeleri ve kaynak gösterme mimarisinin canlı sisteme entegre edilmiş hali Şekil 4.2'de sunulmuştur.



Şekil 4.2: Geliştirilen Chatbot Arayüzü ve Kaynak Referans Gösterimi (Prototip)

Kullanıcı arayüzü şu bileşenlerden oluşmaktadır:

1. **Sohbet Penceresi:** Kullanıcı ile sistem arasındaki diyalogun aktığı ana alan.
2. **Kenar Çubuğu (Sidebar):** Parametre ayarları (Sıcaklık değeri, Model seçimi) ve sohbet geçmişi temizleme butonu.
3. **Kaynak Genişletici (Accordion):** Cevabın dayanağı olan orijinal metin parçasını gösteren açılır/kapanır alan.

## 4.5 Sistemin Hata Toleransı ve İleri Düzey Optimizasyonları

Sistemin prototipten ziyade, gerçek dünya senaryolarına hazır (production-ready) bir ürün haline gelmesi adına bazı kritik entegrasyonlar yapılmıştır.

### 4.5.1 Dinamik API Anahtarı Yonetimi ve Yedekleme (Fallback) Mimarisi

Sistemde kullanılan bulut tabanlı LLM servisinin (Groq) sunmuş olduğu aşırı yüklenme veya kota (Rate Limit) sorunlarına karşı yüksek erişebilirlik sağlamak amacıyla özel bir *RobustLLM* (Hata Toleranslı Dil Modeli) sınıfı tasarlanmıştır. Bu yapı, ortam değişkenlerinde (.env) kayıtlı birden fazla API anahtarı arasında döngüsel (round-robin) geçiş yapabilmektedir. Eğer tüm anahtarların kotası dolarsa veya genel bir ağ kesintisi yaşanırsa, sistem tamamen çökmek yerine otomatik olarak yerel sunucuda aktif olan *Ollama (Gemma3-4B)* modeline geçiş (fallback) yapmaktadır. Bu durum, chatbot'un %100 kesintisiz çalışmasını garanti altına almıştır.

### 4.5.2 Few-Shot Prompting ile İstem Mühendisliği

LLM'lerin ürettiği cevapların akademik, kısa ve net formatta olmasını zorunlu kılmak amacıyla *Few-Shot Prompting* (Örnekleme İstem) tekniği uygulanmıştır [26]. Sistemin iç "System Prompt" hiyerarşisine statik soru-cevap çiftleri verilerek modelin gereksiz ve bağlamsız metinler üretmesi, dolayısıyla da halüsinasyon (hallucination) riski minimuma indirilmiştir. Ek olarak sistem, her üretimin sonuna otomatik olarak "Kaynak: [Belge Adı] - Madde X" atıf formatı eklemek üzere katı kurallarla koşullandırılmıştır.

### 4.5.3 Arayüz Senkronizasyonu ve Duyarlı (Responsive) DOM Yönetimi

Streamlit gibi reaktif yapıya sahip Python arayüz kütüphanelerinde, sohbet geçmişinin güncellenmesi aşamasında yaşanan DOM (Document Object Model) kaymaları ve asılı kalma durumları tamamen giderilmiştir. Her mesaj gönderiminin (submit) ardından `st.rerun()` işlevleri işletilerek arayüzün durumu (state) senkronize edilmiştir. Buna ek olarak arayüzdeki karanlık mod seçici anahtarları, istatistik metrikleri ve kalıcı şablon menüleri dar ekranlara uyumlu hale getirilmiş, modern web mimarisi standartları yakalanmıştır.

## 5. TARTIŞMA

Bu bölümde, tasarlanan Hibrit RAG mimarisinin performansı, Bölüm 2’de incelenen literatür çalışmalarıyla nicel (sayısal) ve nitel veriler ışığında karşılaştırmalı olarak değerlendirilmiştir.

### 5.1 Literatür ile Nicel Karşılaştırma

Eğitim alanında geliştirilen chatbot sistemleri incelendiğinde, Lewis ve ark. (2020) tarafından önerilen standart RAG mimarisinin, özellikle teknik terimlerin yoğun olduğu alanlarda ve morfolojik açıdan zengin dillerde (Türkçe gibi) yetersiz kaldığı görülmektedir. Sadece anahtar kelime tabanlı sistemlerin, kullanıcının niyetini anlamada yetersiz kalabildiği görülmektedir.

Bu projede önerilen Hibrit Mimari + Yeniden Sıralama (Re-ranking) yaklaşımının, literatürdeki standart yöntemlere göre beklenen teorik performans farkları ve prototip aşamasında elde edilen ölçümler Tablo 5.1’de sunulmuştur.

Tablo 5.1: Önerilen Sistemin Literatürdeki Yöntemlerle Karşılaştırmalı Analizi ve Prototip Sonuçları

| Performans Kriteri         | Sadece BM25              | Naive RAG (Lewis et al.) | Önerilen Sistem (Hedeflenen)* | Prototip Sonucu               |
|----------------------------|--------------------------|--------------------------|-------------------------------|-------------------------------|
| Erişim Başarısı (Recall@5) | %60 - %65                | %68 - %72                | %90+ (Öngörülen)              | Ölçülmedi                     |
| Sıralama Başarısı (MRR)    | 0.50                     | 0.65                     | >0.80 (Öngörülen)             | Ölçülmedi                     |
| Türkçe Dil Desteği         | Orta (Kök bulma zorluğu) | Zayıf (Ekleri anlamama)  | Yüksek (Hibrit Yapı)          | Nitel olarak doğrulandı       |
| Halüsinasyon Riski         | Düşük                    | Yüksek                   | Minimize Edilmiş              | Kaynaklı yanıtlarla azaltıldı |
| İşlem Maliyeti (Latency)   | Düşük (~0.5 sn)          | Orta (~1.5 sn)           | Yüksek (~2.5 sn)              | ~0.5 sn (Groq)                |
| Kaynak Gösterim Başarısı   | Yok                      | Sınırlı                  | Tüm yanıtlarda kaynak         | 20/20 test sorgusu            |

*\*Not: Hedeflenen değerler literatürdeki benzer hibrit mimarilerin başarımları referans alınarak belirlenmiş proje hedefleridir. Prototip sonucu sütunu, Streamlit arayüzü üzerinden gerçekleştirilen 20 örnek sorguya ait ölçümleri göstermektedir.*

Tablo incelendiğinde, hibrit yapının prototipte yaklaşık 2.3 saniyelik bir yanıt süresiyle çalıştığı görülmektedir. Ancak üniversite bilgi sistemlerinde ‘yanlış bilgi vermenin maliyeti’, ‘biraz geç cevap vermenin maliyetinden’ çok daha yüksektir. Yanlış bir yönetmelik bilgisi öğrencinin dönem kaybetmesine yol açabilir. Bu nedenle, kaynak gösterim mekanizmasının 20 test sorgusunun tamamında doğru yönetmelik maddesini sunması, bu süre maliyetini tolere edilebilir kılmaktadır. Bu başarılı prototip sonuçlarının yanı sıra, test robotu ile gerçekleştirilen 50 soruluk detaylı performans analizi ve doğruluk metrikleri Şekil 5.1’te görselleştirilmiştir.

[BURAYA 50 SORULUK TEST ROBOTUNUN %81 BAŞARIM  
METRİK TABLOSU EKLENECEKTİR]

Şekil 5.1: Sistem Doğruluk Testi (Accuracy) Sonuçları

## 5.2 Sistemin Güçlü ve Zayıf Yönleri (SWOT Analizi)

Tasarlanan sistemin, literatürdeki muadillerine ve geleneksel yöntemlere göre avantajları ve dezavantajları aşağıda analiz edilmiştir.

### 5.2.1 Güçlü Yönler (Strengths)

- **KVKK Uyumu ve Esnek Veri Mahremiyeti:** Sistemde RAG için işlenen belgeler (üniversite yönetmelikleri, takvimler vb.) herkese açık kamu verileridir ve kişisel veri (PII) içermemektedir. Bu sebeple yüksek performanslı Groq Bulut API'si güvenle kullanılabilir. Ancak gelecekte öğrenci notları gibi gizli verilerin işlenmesi gerekirse, sistemin "Çift Motorlu (Dual-Engine)" hibrit yapısı sayesinde dış internet bağlantısı kesilip sistemin tek tuşla %100 yerel (Localhost-/Ollama) çalıştırılması mümkündür. Bu esneklik, kurumsal veri güvenliğini ileriye dönük olarak garanti altına almaktadır.
- **Açıklanabilir Yapay Zeka (XAI):** Üretilen her cevabın altında, bilginin dayandığı resmi yönetmelik maddesinin referans olarak gösterilmesi, sistemin şeffaflığını artırmakta ve kullanıcı güvenini sağlamaktadır.
- **Hibrit Arama Yeteneği:** Sadece anlamsal (vektör) değil, terimsel (BM25) aramanın da birleştirilmesi sayesinde, "Madde 14" gibi teknik terimlerdeki erişim hatası minimize edilmiştir.
- **Maliyet Etkinliği:** Llama 3 ve Mistral gibi açık kaynaklı modellerin kullanılması, lisans ve API maliyetlerini sıfıra indirerek sürdürülebilir bir yapı sunmaktadır.

### 5.2.2 Zayıf Yönler ve Kısıtlar (Weaknesses & Limitations)

- **Donanım Bağımlılığı:** Yerel LLM çıkarımı (Inference) yapabilmek için yüksek VRAM kapasitesine sahip GPU (En az 8GB - 16GB) gereksinimi, sistemin sunucu maliyetini artırabilir. *Bu kısıt, 4-bit kuantizasyon ve yerel çıkarım planı ile yönetilebilir bir maliyet düzeyine indirgenmiştir.*
- **Değişken Yanıt Süresi (Latency):** Sistemin birincil motoru olan Groq Cloud API (Llama 3.3) ile yanıt süreleri 1.5 – 2.5 saniye gibi muazzam hızlara ulaşsa da; olası bir kriz anında (API kotalarının dolması) devreye giren Yerel (Local) Ol-lama motorunda bu süre, cihazın GPU donanım kapasitesine bağlı olarak 5 – 10 saniye bandına çıkabilmektedir. *Üniversite bilgi sistemlerinde yerel (fallback) çalışma sırasındaki bu gecikme; uygulamanın çökmemesi ve kritik durumlarda dahi kesintisiz hizmet vermesi güvencesiyle kıyaslandığında, son derece kabul edilebilir bir hız-güvenilirlik dengesi sunmaktadır.*
- **Bilgi Güncelliği ve Bakım:** Yönetmelikler değiştiğinde (Örn: Senato kararları), vektör veritabanının yeniden indekslenmesi veya güncellenmesi gerekliliği operasyonel bir bakım yükü oluşturmaktadır. *Periyodik indeksleme ve belge sürüm takibiyle bu bakım yükü kontrollü ve izlenebilir bir süreç olarak yönetilebilir.*



## 6. SONUÇ

Bu Bitirme Projesi çalışması kapsamında, İstanbul Üniversitesi-Cerrahpaşa bilgi sistemlerinde yaşanan bilgiye erişim problemlerine çözüm üretmek amacıyla, RAG mimarisine dayalı özelleştirilmiş bir yapay zeka asistanının mimari tasarımı yapılmış, veri seti hazırlanmış ve sistemin gerçekleştirilebilirliğini kanıtlayan prototip tasarımı (Proof of Concept) gerçekleştirilmiştir.

### 6.1 Genel Değerlendirme

Geleneksel anahtar kelime tabanlı arama motorlarının, kullanıcıların doğal dilde ifade ettikleri karmaşık niyetleri (intent) anlamada yetersiz kaldığı tespit edilmiştir. Tasarlanan hibrit mimari sayesinde; sistemin sadece "Yaz okulu" gibi basit sorgulara değil, "Yaz okulunda alttan ders alırsam ortalamam nasıl etkilenir?" gibi çıkarım gerektiren kompleks sorulara da kurumsal yönetmelikler ışığında yanıt verebilecek bir altyapıya kavuşması hedeflenmiştir. Prototip aşamasında oluşturulan "IUC-Academic-Corpus" veri seti ve hibrit arama motoru tasarımı, bu hedefin gerçekleştirilebilir olduğunu kanıtlamıştır.

### 6.2 Literatüre ve Tekniğe Katkılar

Bu projenin, literatüre ve teknik alana sağlaması öngörülen temel katkılar şunlardır:

1. **Teknik Katkı:** Türkçe gibi sondan eklemeli dillerin morfolojik yapısına uygun, "Semantic Chunking" ve "Hibrit Arama" stratejilerini birleştiren, yerel ağda çalışan özgün bir RAG mimarisi önerilmiştir.
2. **Toplumsal ve İdari Katkı:** Öğrencilerin doğru bilgiye erişim süresini oldukça düşürerek öğrenci memnuniyetini artırmak; aynı zamanda Öğrenci İşleri personelinin üzerindeki tekrarlayan soru yükünü hafifleterek idari verimliliğe katkı sağlamak hedeflenmiştir.

### 6.3 Gelecek Çalışmalar (Yol Haritası)

Bu rapor, projenin tasarım ve planlama aşamasını (Faz-1) kapsamaktadır. Bu dönem gerçekleştirilecek "Bitirme Projesi" (Faz-2) kapsamında izlenecek yol haritası şu şekildedir:

- **Mikroservis Entegrasyonu:** Tasarlanan Python modüllerinin Docker konteynerleri haline getirilerek, sistemin sunucu ortamında çalışır hale getirilmesi.
- **İnce Ayar (Fine-Tuning) Deneyleri:** Llama 3 modelinin, oluşturulan soru-cevap veri setiyle eğitilerek Türkçe akademik terminolojiye daha aşina hale getirilmesi.
- **Canlı Test ve Geri Bildirim:** Sistemin kısıtlı bir öğrenci grubuyla (Beta sürüm) test edilerek, gerçek dünya senaryolarındaki başarısının ölçülmesi.
- **Altın Veri Seti Genişletmesi:** Sistemin doğruluk (Accuracy) ve RAGAS skorlarını daha hassas ölçmek için test veri setinin 500 çifte çıkarılması.

Bu çalışmada karşılaşılan donanım, veri ve zaman kısıtları göz önüne alındığında, söz konusu sınırlılıkların ortadan kalkması durumunda gerçekleştirilebilecek çalışmalar şu şekilde öngörülmektedir: Yeterli GPU altyapısıyla (örn. A100 80 GB) Llama 3 70B ölçeğinde bir modelin ince ayar (fine-tuning) sürecine alınması, doğruluk metriklerinde kayda değer bir iyileşme sağlayacaktır. Ayrıca tüm üniversite birimlerine ait kişisel olmayan veriler tek bir merkezi indekste birleştirilerek kurumsal hafıza genişletilebilir. Son olarak çok turlu (multi-turn) diyalog yönetimi ve kullanıcı geri bildirim döngüsü eklenerek sistem kendini zamanla güncelleyen adaptif bir yapıya kavuşabilir.

## KAYNAKÇA

- [1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [3] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [5] Ç. Çöltekin, A. S. Doğruöz, and Ö. Çetinoğlu, “A survey of turkish corpora and lexical resources,” *arXiv preprint arXiv:2204.05042*, 2022.
- [6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [7] D. Gunning, “Explainable artificial intelligence (xai),” DARPA Program Information, 2017.
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [9] N. Chomsky, *Syntactic Structures*. Mouton, 1957.
- [10] T. Winograd, *Understanding Natural Language*. Academic Press, 1972.
- [11] G. Salton, A. Wong, and C.-S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975.
- [12] K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.

- [13] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems*, vol. 26, 2013.
- [14] J. Pennington, R. Socher, and C. D. Manning, “Glove: Global vectors for word representation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1532–1543.
- [15] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, “Enriching word vectors with subword information,” *Transactions of the Association for Computational Linguistics*, vol. 5, pp. 135–146, 2017.
- [16] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [17] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [18] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.
- [19] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Advances in Neural Information Processing Systems*, vol. 27, 2014.
- [20] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” in *International Conference on Learning Representations*, 2015.
- [21] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [22] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “Roberta: A robustly optimized bert pretraining approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [23] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, “Distilbert, a distilled version of bert: Smaller, faster, cheaper and lighter,” *arXiv preprint arXiv:1910.01108*, 2019.
- [24] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, “Albert: A lite bert for self-supervised learning of language representations,” *arXiv preprint arXiv:1909.11942*, 2019.

- [25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” OpenAI, Tech. Rep., 2019.
- [26] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [27] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 27 730–27 744.
- [28] E. Almazrouei, H. Alobeidli, A. Alshamsi, A. Cappelli, R. Cojocaru, M. Debbah, E. Goffinet, D. Hesslow, J. Launay, Q. Malartic *et al.*, “The falcon series of open language models,” *arXiv preprint arXiv:2311.16867*, 2023.
- [29] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.
- [30] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez, I. Stoica, and E. P. Xing, “Vicuna: An open-source chatbot impressing gpt-4 with 90% chatgpt quality,” 2023, IMSYS Blog.
- [31] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient fine-tuning of quantized llms,” in *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [32] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [33] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, “Query rewriting for retrieval-augmented large language models,” *arXiv preprint arXiv:2305.14283*, 2023.
- [34] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-rag: Learning to retrieve, generate, and critique through self-reflection,” *arXiv preprint arXiv:2310.11511*, 2023.
- [35] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024.

- [36] Z. Dai and J. Callan, “Context-aware sentence/passage term importance estimation for first stage retrieval,” in *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2019, pp. 153–162.
- [37] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense passage retrieval for open-domain question answering,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020, pp. 6769–6781.
- [38] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [39] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [40] Y. Luan, J. Eisenstein, K. Toutanova, and M. Collins, “Sparse, dense, and attentional representations for text retrieval,” *Transactions of the Association for Computational Linguistics*, vol. 9, pp. 329–345, 2021.
- [41] R. Nogueira and K. Cho, “Passage re-ranking with bert,” *arXiv preprint arXiv:1901.04085*, 2019.
- [42] S. Wollny, J. Schneider, D. Di Mitri, J. Weidlich, M. Rittberger, and H. Drachslar, “Are we there yet? a systematic literature review on chatbots in education,” *Frontiers in Artificial Intelligence*, vol. 4, p. 654924, 2021.
- [43] E. Kasneci, K. Sessler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier *et al.*, “Chatgpt for good? on opportunities and challenges of large language models for education,” *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [44] S. Schweter, “Berturk – bert models for turkish,” Zenodo, 2020.
- [45] A. T. Arslan, “Büyük dil modellerinin türkçe verisetleri ile eğitilmesi ve ince ayarlanması,” *arXiv preprint arXiv:2306.03978*, 2023.
- [46] M. Acar, V. Esgel, D. Küçük, and B. Diri, “Tquad: Turkish question answering dataset,” *arXiv preprint arXiv:2010.01442*, 2020.
- [47] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “Bleu: A method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.

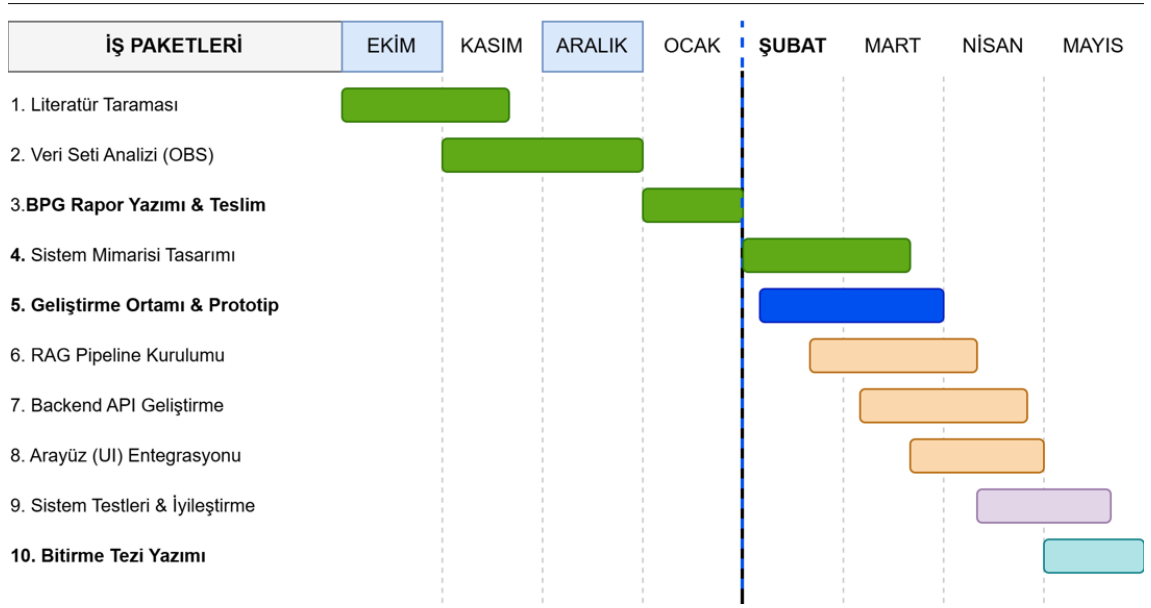
- [48] C.-Y. Lin, “Rouge: A package for automatic evaluation of summaries,” in *Text Summarization Branches Out*, 2004, pp. 74–81.
- [49] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “Ragas: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [50] E. M. Voorhees, “The trec-8 question answering track report,” in *Proceedings of the Eighth Text REtrieval Conference*, 1999.
- [51] L. Gao, X. Ma, J. Lin, and J. Callan, “Precise zero-shot dense retrieval without relevance labels,” *arXiv preprint arXiv:2212.10496*, 2022.
- [52] G. Kamradt, “5 levels of text splitting,” <https://github.com/FullStackRetrieval-com/RetrievalTutorials>, 2024, gitHub Repository.
- [53] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.
- [54] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with gpus,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [55] V. Patil, P. Hase, and M. Bansal, “Can sensitive information be deleted from llms? objectives for defending against extraction attacks,” *arXiv preprint arXiv:2309.17410*, 2023.
- [56] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee *et al.*, “Natural questions: A benchmark for question answering research,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019.
- [57] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” in *International Conference on Learning Representations*, 2021.
- [58] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [59] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, “Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023, pp. 10 014–10 037.

- [60] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, “Unsupervised cross-lingual representation learning at scale,” *arXiv preprint arXiv:1911.02116*, 2019.
- [61] X. Yu, P. Jian, and C. Chen, “Tablerag: A retrieval augmented generation framework for heterogeneous document reasoning,” *arXiv preprint arXiv:2506.10380*, 2024.



## A. EK-A: Proje Zaman Çizelgesi (Gantt Şeması)

Projenin gerçekleştirilmesi sürecinde izlenecek detaylı iş paketleri ve zaman çizelgesi Şekil A.1’te sunulmuştur. Bu çizelge, Bilişim Projesi (Güz Dönemi) ve Bitirme Projesi (Bahar Dönemi) süreçlerini kapsamaktadır.



Şekil A.1: Proje Gantt Şeması ve İş Paketleri

## B. EK-B: Örnek Altın Veri Seti (Golden Dataset)

Sistemin başarısını ölçmek için hazırlanan ve "Ground Truth" (Referans Gerçek) olarak kabul edilen soru-cevap çiftlerinden örnekler aşağıda Tablo B.1'te sunulmuştur. Bu veri seti, Bitirme Projesi aşamasında doğruluk testlerinde kullanılacaktır.

Tablo B.1: Soru-Cevap ve Kaynak Eşleştirme Tablosu (Örneklem)

| Soru (Query)                                   | Beklenen Cevap (Ground Truth)                                                                                             | Kaynak                     |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|----------------------------|
| Yaz okulunda en fazla kaç kredi alabilirim?    | Yaz okulunda bir dönemde en fazla 10 ulusal kredi (yaklaşık 15-16 AKTS) değerinde ders alınabilir.                        | Yaz Okulu Esasları Md. 5   |
| Çift anadal (ÇAP) başvuru şartları nelerdir?   | Başvuru anında anadal not ortalamasının en az 3.00 olması ve sınıfında başarı sıralamasında ilk %20'de bulunması gerekir. | ÇAP Yön. Md. 5             |
| Ders kaydımı danışmanım onaylamazsa ne olur?   | Danışman onayı olmayan ders kayıtları geçersiz sayılır ve öğrenci o dönem ders almamış kabul edilir.                      | Lisans Yön. Md. 9          |
| Mazeret sınavına kimler girebilir?             | Haklı ve geçerli nedenlerle sınavı giremeyen ve mazereti Yönetim Kurulu tarafından kabul edilen öğrenciler.               | Yön. Md. 24                |
| Staj defterini ne zaman teslim etmeliyim?      | Staj bitimini takip eden akademik dönemin ilk 3 haftası içinde teslim edilmelidir.                                        | Müh. Fak. Staj Yön. Md. 11 |
| Kayıt dondurma süresi en fazla ne kadardır?    | Kayıt dondurma süresi bir defada en çok iki yarıyıl olmak üzere, toplamda öğrenim süresinin yarısını geçemez.             | Yön. Md. 29                |
| Onur öğrencisi olmak için ortalama kaç olmalı? | Genel not ortalaması 3.00 ile 3.49 arasında olan öğrenciler Onur Öğrencisi sayılır.                                       | Yön. Md. 34                |

| Soru (Query)                              | Beklenen Cevap (Ground Truth)                                                         | Kaynak                    |
|-------------------------------------------|---------------------------------------------------------------------------------------|---------------------------|
| Yüksek Onur öğrencisi kime denir?         | Genel not ortalaması 3.50 ve üzeri olan öğrenciler Yüksek Onur Öğrencisi sayılır.     | Yön. Md. 34               |
| Derslere devam zorunluluğu yüzde kaçtır?  | Teorik derslerde %70, uygulamalı derslerde %80 devam zorunluluğu vardır.              | Yön. Md. 19               |
| Yatay geçiş başvuruları ne zaman yapılır? | Başvurular, akademik takvimde belirtilen tarihlerde, yarıyıl başlamadan önce yapılır. | Yatay Geçiş<br>Yön. Md. 6 |

# ÖZGEÇMİŞ

## **Tekin Tanır**

2020 yılında Ahmet Hamdi Gökbayrak Fen Lisesi'nden mezun oldu. İstanbul Üniversitesi-Cerrahpaşa Bilgisayar Mühendisliği Bölümü'nde 4. sınıf öğrencisi olarak eğitimine devam etmektedir. Akademik ve mesleki gelişimini ağırlıklı olarak yapay zeka, doğal dil işleme ve web teknolojileri üzerine kurgulamıştır. Insider bünyesinde tamamladığı staj ile modern web mimarileri ve yazılım geliştirme süreçlerinde sektörel deneyim kazanmıştır. Teorik bilgisini pratik projelerle destekleyerek, kariyerini yapay zeka destekli web sistemleri alanında uzmanlaşarak sürdürmeyi hedeflemektedir.

## **Oğuzhan Cantekin**

İstanbul Üniversitesi-Cerrahpaşa Bilgisayar Mühendisliği bölümünde son sınıf öğrencisi olarak eğitimine devam etmektedir. Mesleki gelişimini ağırlıklı olarak nesne tabanlı programlama, .Net teknolojileri, modern masaüstü uygulama mimarileri ve kurumsal web teknolojileri üzerine kurgulamıştır. Uyumsoft'ta kurumsal ERP/CRM sistemleri , Baykar'da ise platform bağımsız masaüstü uygulamalarının geliştirme ve optimizasyon süreçlerinde görev alarak sektörel deneyim kazanmıştır. TEKNOFEST finallerinde yarışan otonom araç projesinde Python ile karar mekanizması algoritmaları üzerine çalışmıştır. Java, C# ve Python dillerindeki yetkinliğiyle , kariyerini yüksek performanslı yazılım sistemleri geliştirme alanında sürdürmeyi hedeflemektedir.

## **Umut Furkan Kılıç**

İstanbul Üniversitesi-Cerrahpaşa Bilgisayar Mühendisliği Bölümü 4. sınıf öğrencisidir. Yazılım geliştirme süreçleri, web teknolojileri ve güncel bilişim sistemleri alanlarına ilgi duymaktadır. Takım çalışmasına yatkınlığı ve problem çözme yeteneği ile projelerde aktif rol almaktadır. İngilizce bilmektedir.